

Section-Based – Development with AWS Services (CDA)

1. Question

Category: CDA – Development with AWS Services

A company has an application that is using CloudFront to serve their static contents to their users around the globe. They are receiving a number of bad reviews from their customers lately because it takes a lot of time to log into their website. Sometimes, their users are also getting HTTP 504 errors which is why the developer was instructed to fix this problem immediately.

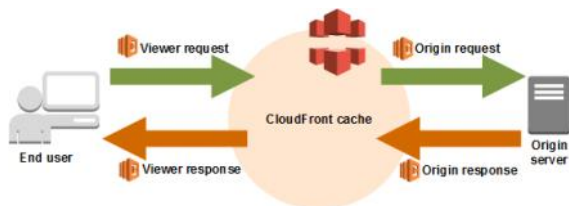
Which of the following combination of options should the developer use together to set up a cost-effective solution for this scenario? (Select TWO.)

- Launch your application to multiple AWS regions to serve your global users. Use a Route 53 record with latency routing policy to route incoming traffic to the region with the best latency to the user.
- Launch your application to multiple and geographically disperse VPCs on various AWS regions then create a transit VPC to easily connect all your resources. Use several Lambda functions in each region using the AWS Serverless Application Model (SAM) service to improve the overall application performance.
- Add a Cache-Control max-age directive to your objects in CloudFront and specify the longest practical value for max-age to increase the cache hit ratio of your CloudFront distribution.
- Customize the content that the CloudFront web distribution delivers to your users using Lambda@Edge, which allows your Lambda functions to execute the authentication process in AWS locations closer to the users.
- Configure an origin failover by creating an origin group with two origins. Specify one as the primary origin and the other as the second origin which CloudFront automatically switches to when the primary origin returns specific HTTP status code failure responses.

Incorrect

Lambda@Edge lets you run Lambda functions to customize the content that CloudFront delivers, executing the functions in AWS locations closer to the viewer. The functions run in response to CloudFront events, without provisioning or managing servers. You can use Lambda functions to change CloudFront requests and responses at the following points:

- After CloudFront receives a request from a viewer (viewer request)
- Before CloudFront forwards the request to the origin (origin request)
- After CloudFront receives the response from the origin (origin response)
- Before CloudFront forwards the response to the viewer



In the given scenario, you can use Lambda@Edge to allow your Lambda functions to customize the content that CloudFront delivers and to execute the authentication process in AWS locations closer to the users. In addition, you can set up an origin failover by creating an origin group with two origins with one as the primary origin and the other as the second origin, which CloudFront automatically switches to when the primary origin fails. This will alleviate the occasional HTTP 504 errors that users are experiencing.

Hence, the correct answers in this scenario are:

- Customize the content that the CloudFront web distribution delivers to your users using Lambda@Edge, which allows your Lambda functions to execute the authentication process in AWS locations closer to the users.
- Configure an origin failover by creating an origin group with two origins. Specify one as the primary origin and the other as the second origin which CloudFront automatically switches to when the primary origin returns specific HTTP status code failure responses.

The option that says: Launch your application to multiple AWS regions to serve your global users. Use a Route 53 record with latency routing policy to route incoming traffic to the region with the best latency to the user is incorrect. Although this may resolve the performance issue, this solution entails a significant implementation cost since you have to deploy your application to multiple AWS regions. Remember that the scenario asks for a solution that will improve the performance of the application with minimal cost.

The option that says: Launch your application to multiple and geographically disperse VPCs on various AWS regions then create a transit VPC to easily connect all your resources. Use several Lambda functions in each region using the AWS Serverless Application Model (SAM) service to improve the overall application performance is incorrect. Although setting up multiple VPCs across various regions which are connected with a transit VPC is valid, this solution still entails higher setup and maintenance costs. A more cost-effective option would be to use Lambda@Edge instead.

The option that says: Add a Cache-Control max-age directive to your objects in CloudFront and specify the longest practical value for max-age to increase the cache hit ratio of your CloudFront distribution is incorrect because improving the cache hit ratio for the CloudFront distribution is irrelevant in this scenario. You can improve your cache performance by increasing the proportion of your viewer requests that are served from CloudFront edge caches instead of going to your origin servers for content. However, take note that the problem in the scenario is the slow authentication process of your global users and not just the caching of the static objects.

References:

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/high_availability_origin_failover.html
<https://docs.aws.amazon.com/lambda/latest/dg/lambda-edge.html>

Check out these Amazon CloudFront and AWS Lambda Cheat Sheets:

<https://tutorialsdojo.com/amazon-cloudfront/>
<https://tutorialsdojo.com/aws-lambda/>

2. Question

Category: CDA – Development with AWS Services

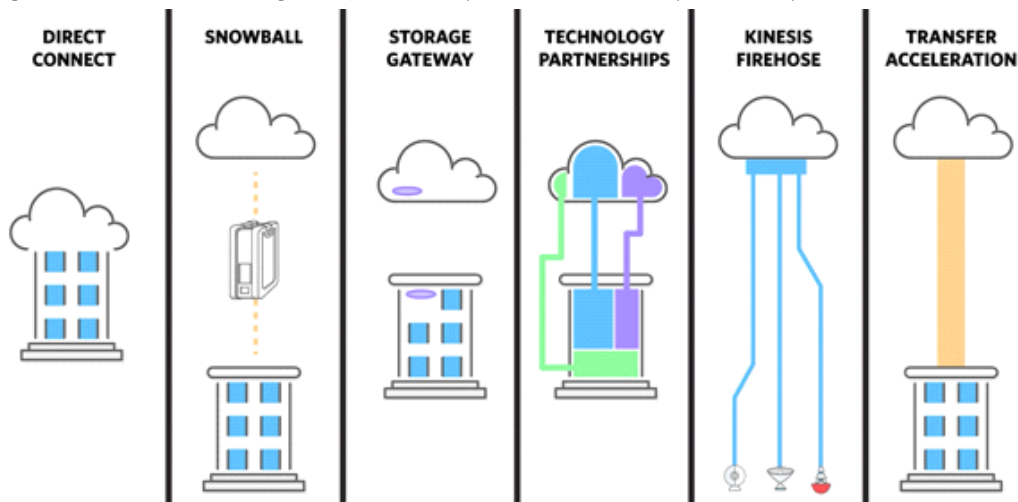
A global financial company has hundreds of users from all over the world who regularly upload terabytes of transactional data to a centralized Amazon S3 bucket. Users from different parts of the globe are experiencing delays in uploading data, which in turn affects processing times. The goal is to improve data throughput and ensure consistently fast data transfer to the S3 bucket regardless of the user's location.

Which of the following should be used to satisfy the above requirement?

- S3 Transfer Acceleration
- Amazon CloudFront
- AWS Direct Connect
- AWS Transfer for SFTP

Incorrect

Amazon S3 Transfer Acceleration enables fast, easy, and secure transfers of files over long distances between your client and your Amazon S3 bucket. Transfer Acceleration leverages Amazon CloudFront's globally distributed AWS Edge Locations. As data arrives at an AWS Edge Location, data is routed to your Amazon S3 bucket over an optimized network path.



Hence, the correct answer is: **S3 Transfer Acceleration**.

AWS Transfer for SFTP is incorrect because this is just a fully managed service that enables the transfer of files directly into and out of Amazon S3 using the Secure File Transfer Protocol (SFTP) which is also known as Secure Shell (SSH) File Transfer Protocol. It does not provide a fast, easy, and secure way to transfer files over long distances between your client and your Amazon S3 bucket.

AWS Direct Connect is incorrect because you have users all around the world and not just on your on-premises data center. Direct Connect would be too costly and is definitely not suitable for this purpose.

Amazon CloudFront is incorrect because this service is primarily used to serve static content and not as a transfer accelerator going to or from Amazon S3. CloudFront is a fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers globally with low latency and high transfer speeds.

References:

<http://docs.aws.amazon.com/AmazonS3/latest/dev/transfer-acceleration.html>

<https://docs.aws.amazon.com/AmazonS3/latest/dev/transfer-acceleration-examples.html>

Check out this Amazon S3 Cheat Sheet:

<https://tutorialsdojo.com/amazon-s3/>

Comparison of AWS Services Cheat Sheets:

<https://tutorialsdojo.com/comparison-of-aws-services/>

3. Question

Category: CDA – Development with AWS Services

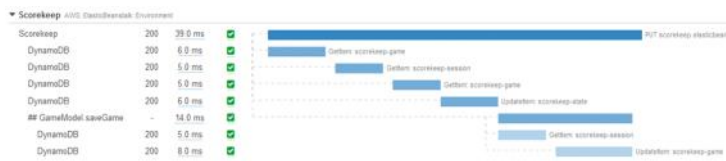
In order to quickly troubleshoot their systems, your manager instructed you to record the calls that your application makes to all AWS services and resources. You developed a custom code that will send the segment documents directly to X-Ray by using the `PutTraceSegments` API.

What should you include in your segment document to meet the above requirement?

- metadata
- tracing header
- annotations
- subsegments

Incorrect

A segment can break down the data about the work done into **subsegments**. Subsegments provide more granular timing information and details about downstream calls that your application made to fulfill the original request. A subsegment can contain additional details about a call to an AWS service, an external HTTP API, or an SQL database. You can even define arbitrary subsegments to instrument specific functions or lines of code in your application.



For services that don't send their own segments like Amazon DynamoDB, X-Ray uses subsegments to generate *inferred segments* and downstream nodes on the service map. This lets you see all of your downstream dependencies, even if they don't support tracing, or are external.

Subsegments represent your application's view of a downstream call as a client. If the downstream service is also instrumented, the segment that it sends replaces the inferred segment generated from the upstream client's subsegment. The node on the service graph always uses information from the service's segment, if it's available, while the edge between the two nodes uses the upstream service's subsegment.

Hence, the correct answer in this scenario is to include **subsegments** in your segment document.

Including **tracing header** is incorrect because this is added in the HTTP request header and not on the segment document. A tracing header (`X-Amzn-Trace-Id`) can originate from the X-Ray SDK, an AWS service, or the client request.

Including **metadata** is incorrect because this does not record the calls to AWS services and resources that are made by the application. Segments and subsegments can include a `metadata` object containing one or more fields with values of any type, including objects and arrays.

Including **annotations** is incorrect because just like *metadata*, this also does not record the application's calls to your AWS services and resources. Segments and subsegments can include an `annotations` object containing one or more fields that X-Ray indexes for use with filter expressions.

References:

<https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html>

<https://docs.aws.amazon.com/xray/latest/devguide/xray-api-segmentdocuments.html#api-segmentdocuments-subsegments>

Check out this AWS X-Ray Cheat Sheet:

<https://tutorialsdojo.com/aws-x-ray/>

Instrumenting your Application with AWS X-Ray:

<https://tutorialsdojo.com/instrumenting-your-application-with-aws-x-ray/>

4. Question

Category: CDA – Development with AWS Services

A company is transitioning their systems to AWS due to the limitations of their on-premises data center. As part of this project, a developer was assigned to build a brand new serverless architecture in AWS, which will be composed of AWS Lambda, API Gateway, and DynamoDB in a single stack. She needs a simple and reliable framework that will allow her to share configuration such as memory and timeouts between resources and deploy all related resources together as a single, versioned entity.

Which of the following is the MOST appropriate service that the developer should use in this scenario?

- Serverless Application Framework
- AWS CloudFormation
- AWS SAM
- AWS Systems Manager

Incorrect

The AWS Serverless Application Model (AWS SAM) is an open source framework for building serverless applications. It provides shorthand syntax to express functions, APIs, databases, and event source mappings. You define the application you want with just a few lines per resource and model it using YAML.

AWS SAM is natively supported by AWS CloudFormation and provides a simplified way of defining the Amazon API Gateway APIs, AWS Lambda functions, and Amazon DynamoDB tables needed by your serverless application. During deployment, SAM transforms and expands the SAM syntax into AWS CloudFormation syntax. Then, CloudFormation provisions your resources with reliable deployment capabilities.

Hence, the correct answer is **AWS SAM**.

AWS CloudFormation is incorrect. Although this service can deploy the serverless application to AWS, it is still more appropriate to use AWS SAM instead. AWS SAM can simplify the deployment of the serverless application by deploying all related resources together as a single, versioned entity.

AWS Systems Manager is incorrect because it is more focused on management and operations of AWS resources, such as automation, patching, and configuration, but it is not a deployment or application modeling tool.

Serverless Application Framework is incorrect. Although it is a well-known framework for building and deploying serverless applications into the AWS cloud, this is not an AWS native solution. It also does not allow configuration of DynamoDB databases or API Gateway APIs, unlike AWS SAM.

References:

<https://aws.amazon.com/serverless/sam/>

<https://aws.amazon.com/serverless/developer-tools/>

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/what-is-sam.html>

Check out this AWS Serverless Application Model Cheat Sheet:

<https://tutorialsdojo.com/aws-serverless-application-model-sam/>

5. Question

Category: CDA – Development with AWS Services

You are working as an IT Consultant for a top investment bank in Europe which uses several serverless applications in their AWS account. They just launched a new API Gateway service with a Lambda proxy integration and you were instructed to test out the new API. However, you are getting a `Connection refused` error whenever you use this Invoke URL <http://779protaw8.execute-api.us-east-1.amazonaws.com/tutorialsdojo/> of the API Gateway.

Which of the following is the MOST likely cause of this issue?

- You are not using WebSocket in invoking the API.
- You are not using HTTP/2 in invoking the API.
- You are not using FTP in invoking the API.
- You are not using HTTPS in invoking the API.

Incorrect

All of the APIs created with Amazon API Gateway expose **HTTPS** endpoints only. Amazon API Gateway does not support unencrypted (HTTP) endpoints. By default, Amazon API Gateway assigns an internal domain to the API that automatically uses the Amazon API Gateway certificate. When configuring your APIs to run under a custom domain name, you can provide your own certificate for the domain.



Calling a deployed API involves submitting requests to the URL for the API Gateway component service for API execution, known as `execute-api`. The base URL for REST APIs is in the following format:

https://{restapi_id}.execute-api.{region}.amazonaws.com/{stage_name}/

where `{restapi_id}` is the API identifier, `{region}` is the region, and `{stage_name}` is the stage name of the API deployment.

Hence, the most likely cause of the issue in the scenario is that **you are not using HTTPS in invoking the API**.

The option that says: **you are not using HTTP/2 in invoking the API** is incorrect because API Gateway only supports HTTPS.

The option that says: **you are not using FTP in invoking the API** is incorrect because API Gateway is using HTTPS to expose the APIs. FTP is primarily used for accessing file servers and not Web APIs.

The option that says: **you are not using WebSocket in invoking the API** is incorrect because all of the APIs created with Amazon API Gateway expose HTTPS endpoints only.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-call-api.html>

<https://aws.amazon.com/api-gateway/faqs/>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

6. Question

Category: CDA – Development with AWS Services

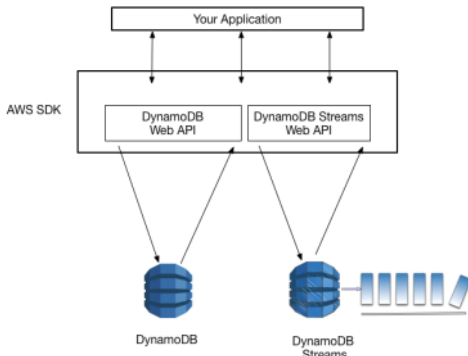
A developer is working on an application which stores data to an Amazon DynamoDB table with the DynamoDB Streams feature enabled. He set up an event source mapping with DynamoDB Streams and AWS Lambda function to monitor any table changes then store the original data of the overwritten item in S3. When an item is updated, it should only send a copy of the item's previous value to an S3 bucket and maintain the new value in the DynamoDB table.

Which `StreamViewType` is the MOST suitable one to use in the DynamoDB configuration to fulfill this scenario?

- `KEYS_ONLY`
- `NEW_IMAGE`
- `OLD_IMAGE`
- `NEW_AND_OLD_IMAGES`

Incorrect

DynamoDB Streams provides a time-ordered sequence of item level changes in any DynamoDB table. The changes are de-duplicated and stored for 24 hours. Applications can access this log and view the data items as they appeared before and after they were modified, in near real time.



Amazon DynamoDB is also integrated with AWS Lambda so that you can create *triggers*—pieces of code that automatically respond to events in DynamoDB Streams. With triggers, you can build applications that react to data modifications in DynamoDB tables.

When an item in the table is modified, `StreamViewType` determines what information is written to the stream for this table. Valid values

for `StreamViewType` are `KEYS_ONLY`, `NEW_IMAGE`, `OLD_IMAGE`, and `NEW_AND_OLD_IMAGES`.

For the `OLD_IMAGE` type, the entire item which has the previous value as it appeared before it was modified is written to the stream. Hence, this is the correct answer in this scenario.

`KEYS_ONLY` is incorrect because it will only write the key attributes of the modified item to the stream. This choice is wrong since the question states that values should be copied as well.

`NEW_IMAGE` is incorrect because it will configure the stream to write the entire item with its new value as it appears after it was modified. This choice is wrong since the stream should capture the item's pre-modified values.

`NEW_AND_OLD_IMAGES` is incorrect because although it writes the new values of the item in the stream, it also includes the old one as well. Since this type will send both the new and the old item images of the item to the stream, this option is wrong. Remember that it should only send a copy of the item's previous value to the S3 bucket, and not the new value in the DynamoDB table. The most suitable one to use here is the `OLD_IMAGE` type.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>

https://docs.aws.amazon.com/en_us/amazondynamodb/latest/developerguide/Streams.Lambda.html

https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_StreamSpecification.html

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

AWS Lambda Integration with Amazon DynamoDB Streams:

<https://tutorialsdojo.com/aws-lambda-integration-with-amazon-dynamodb-streams/>

Tutorials Dojo's AWS Certified Developer Associate Exam Study Guide:

<https://tutorialsdojo.com/aws-certified-developer-associate/>

7. Question

Category: CDA – Development with AWS Services

A company decided to re-use the same Lambda function for multiple stages of their API, but the function should read data from a different Amazon DynamoDB table depending on which stage is being called. In order to accomplish this, they instructed the developer to pass configuration parameters to a Lambda function through mapping templates in API Gateway.

Which of the following is the MOST suitable solution that the developer should use to meet this requirement?

- Create environment variables in the Lambda function.

- Set up an API Gateway Private Integration to the Lambda function.
- Use Stage Variables.
- Set up traffic shifting with Lambda Aliases.

Incorrect

Stage variables are name-value pairs that you can define as configuration attributes associated with a deployment stage of a REST API. They act like environment variables and can be used in your API setup and mapping templates. For example, you can define a stage variable in a stage configuration, and then set its value as the URL string of an HTTP integration for a method in your REST API. Later, you can reference the URL string using the associated stage variable name from the API setup. This way, you can use the same API setup with a different endpoint at each stage by resetting the stage variable value to the corresponding URLs. You can also access stage variables in the mapping templates, or pass configuration parameters to your AWS Lambda or HTTP backend.

With deployment stages in API Gateway, you can manage multiple release stages for each API, such as alpha, beta, and production. Using stage variables you can configure an API deployment stage to interact with different backend endpoints. For example, your API can pass a GET request as an HTTP proxy to the backend web host (for example, <http://tutorialsdodojo.com>). In this case, the backend web host is configured in a stage variable so that when developers call your production endpoint, API Gateway calls example.com. When you call your beta endpoint, API Gateway uses the value configured in the stage variable for the beta stage, and calls a different web host (for example, beta.tutorialsdodojo.com). Similarly, stage variables can be used to specify a different AWS Lambda function name for each stage in your API.

You can also use stage variables to pass configuration parameters to a Lambda function through your mapping templates. For example, you may want to re-use the same Lambda function for multiple stages in your API, but the function should read data from a different Amazon DynamoDB table depending on which stage is being called. In the mapping templates that generate the request for the Lambda function, you can use stage variables to pass the table name to Lambda.

Stage variables are not applied to the security definitions section of the API specification. For example, you cannot use different Amazon Cognito user pools for different stages.

Hence, the correct answer in this scenario is to **use stage variables**.

Setting up an API Gateway Private Integration to the Lambda function is incorrect because this is just used to expose your HTTP/HTTPS resources behind an Amazon VPC to allow access to clients outside of your VPC.

Creating environment variables in the Lambda function is incorrect because using an environment variable alone is not enough to meet this requirement. You have to integrate your Lambda function with API Gateway by using a stage variable, along with a proper mapping configuration.

Setting up traffic shifting with Lambda Aliases is incorrect because this is just like a pointer to a specific Lambda function version. By using aliases, you can access the Lambda function which the alias is pointing to, without the caller knowing the specific version the alias is pointing to. Take note that we are not talking about different versions of the Lambda functions here as the scenario explicitly mentioned that we have passed the configuration parameters from API Gateway to the Lambda function.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/stage-variables.html>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/amazon-api-gateway-using-stage-variables.html>
<https://docs.aws.amazon.com/lambda/latest/dg/lambda-configuration.html>

Check out this Amazon API Gateway and AWS Lambda Cheat Sheet:

<https://tutorialsdodojo.com/amazon-api-gateway/>
<https://tutorialsdodojo.com/aws-lambda/>

8. Question

Category: CDA – Development with AWS Services

A developer is designing the cloud architecture of an internal application which will be used by about a hundred employees. She needs to ensure that the architecture is elastic enough to adequately match the supply of resources to the demand while maintaining its cost-effectiveness.

Which of the following services can provide the MOST elasticity to the architecture? (Select TWO.)

- Amazon RDS
- AWS WAF
- Amazon CloudFront
- Amazon EC2 Spot Fleet
- Amazon DynamoDB

Incorrect

In the traditional data center-based model of IT, once the infrastructure is deployed, it typically runs whether it is needed or not, and all the capacity is paid for, regardless of how much it gets used. In the cloud, resources are **elastic**,

meaning they can instantly grow or shrink to match the requirements of a specific application.

Elasticity allows you to match the supply of resources—which cost money—to demand. Because cloud resources are paid for based on usage, matching needs to utilization is critical for cost optimization. Demand includes both

external usage, such as the number of customers who visit a website over a given period, and internal usage, such as an application team using development and test environments.

There are two basic types of elasticity:

1. Time-based
2. Volume-based

Time-based elasticity means turning off resources when they are not being used, such as a development environment that is needed only during business hours. Volume-based elasticity means matching scale to the intensity of

demand, whether that's compute cores, storage sizes, or throughput. By combining monitoring, tagging, and automation, you can get the most value out of your AWS resources and optimize costs.

By taking advantage of volume-based elasticity, you can scale resources to match capacity. The best tool for accomplishing this task is Auto Scaling, which you can use to optimize performance by automatically increasing the number of EC2 instances during demand spikes and decreasing capacity during lulls to reduce costs. Auto Scaling is well-suited for applications that have stable demand patterns and for ones that experience hourly, daily, or weekly variability in usage.

You can also use a combination of ELB and Auto Scaling to maximize the elasticity of your architecture. Beyond Auto Scaling for Amazon EC2, you can use Application Auto Scaling to automatically scale resources for other AWS

services, including:

- Amazon ECS
- Amazon EC2 Spot Fleets
- Amazon EMR clusters
- Amazon AppStream 2.0 stacks and fleets
- Amazon DynamoDB

Target capacity

Total target capacity
Set your total target capacity (number of instances or vCPUs) to launch. If you specified a launch template, you can allocate part of the target capacity as On-Demand. The number of On-Demand Instances always persists, while Spot Instances can be scaled.

1 instances

☐ Include On-Demand base capacity
Allocate part of target capacity as On-Demand instances

☒ **Maintain target capacity**
Automatically replace interrupted Spot Instances

Interruption behavior
Terminate

☒ **Capacity rebalance**
When a rebalance notification is sent to a Spot Instance, Spot Fleet automatically attempts to replace the instance before it is interrupted. [Learn more](#)

Instance replacement strategy
Launch only
Fleet only launches a replacement instance and will not terminate the instance that receives t...

☒ **Set maximum cost for Spot Instances**
Set the maximum amount per hour that you're willing to pay for all the Spot Instances in your fleet

Set your max cost (per hour)
\$

TUTORIALS DOJO

For Amazon EC2 Spot Fleets, it can either launch instances (scale out) or terminate instances (scale in), within the range that you choose, in response to one or more scaling policies.

DynamoDB

Read/write capacity settings Info

Capacity mode

☒ **Provisioned**
Manage and optimize your costs by allocating read/write capacity in advance.

☐ **On-demand**
Simplify billing by paying for the actual reads and writes your application performs.

Read capacity
Auto scaling Info
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.

☒ On
☐ Off

Minimum capacity units: 1 Maximum capacity units: 10 Target utilization (%): 70

Write capacity
Auto scaling Info
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.

☒ On
☐ Off

Minimum capacity units: 1 Maximum capacity units: 10 Target utilization (%): 70

TUTORIALS DOJO

For Amazon DynamoDB, you can dynamically adjust provisioned throughput capacity in response to actual traffic patterns. This enables a table or a global secondary index to increase its provisioned read and write capacity to handle sudden increases in traffic without throttling. When the workload decreases, Application Auto Scaling decreases the throughput so that you don't pay for unused provisioned capacity.

Hence, the correct answers are:

– **Amazon EC2 Spot Fleet.**

– **Amazon DynamoDB.**

Amazon CloudFront is incorrect because this is primarily helpful for scaling out your application. Moreover, the scenario says that the internal application will only be used by about a hundred employees, which clearly doesn't warrant the use of a CDN or CloudFront.

AWS WAF is incorrect because it only improves the security of your architecture and not its elasticity.

Amazon RDS is incorrect. While Amazon RDS offers robust features and scalability, its cost structure can be higher due to the need for continuous instance uptime, storage, and backup costs.

References:

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-fleet.html>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/AutoScaling.html>

Check out these cheat sheets for Amazon EC2 and Amazon DynamoDB:

<https://tutorialsdojo.com/amazon-elastic-compute-cloud-amazon-ec2/>

<https://tutorialsdojo.com/amazon-dynamodb/>

9. Question

Category: CDA – Development with AWS Services

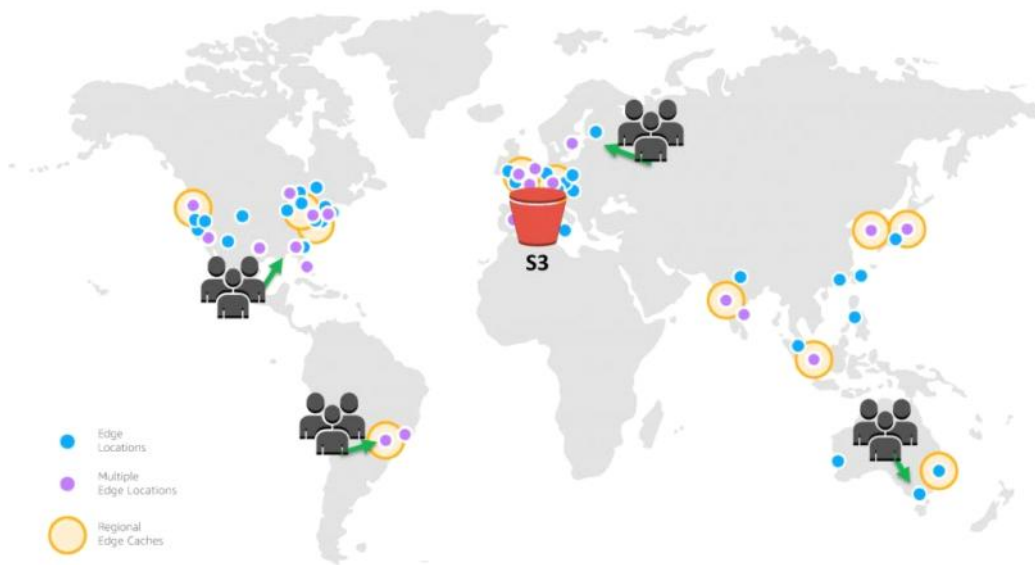
A company is developing a serverless website that consists of images, videos, HTML pages and JavaScript files. There is also a requirement to serve the files with lowest possible latency to its global users.

Which combination of services should be used in this scenario? (Select TWO.)

- Amazon EC2
- Amazon Glacier
- Amazon S3
- Amazon CloudFront
- Amazon Elastic File System

Incorrect

You can configure your application to deliver static content and decrease the end-user latency using Amazon S3 and Amazon CloudFront. High-resolution images, videos, and other static files can be stored in Amazon S3. CloudFront speeds up content delivery by leveraging its global network of data centers, known as edge locations, to reduce delivery time by caching your content close to your end-users.



CloudFront fetches your content from an origin, such as an Amazon S3 bucket, an Amazon EC2 instance, an Amazon Elastic Load Balancing load balancer, or your own web server, when it's not already in an edge location. CloudFront can be used to deliver your entire website or application, including dynamic, static, streaming, and interactive content. You can set your Amazon S3 bucket as the origin of your CloudFront web distribution. Hence, the correct answers are:

– Amazon S3

– Amazon CloudFront

Amazon EC2 is incorrect because EC2 instances are more suited for dynamic, server-side processing and they are region-specific, which means the latency for users would vary depending on their geographical distance from the EC2 instance. They are not designed for low-latency global content delivery of static web assets.

Amazon Elastic File System is incorrect because this is not a suitable service for storing static content, unlike S3. It is a regional service storing data within and across multiple Availability Zones (AZs) for high availability and durability. In addition, you can't directly connect it to CloudFront, unlike S3.

Amazon Glacier is incorrect because this is primarily used for data archival with usually a long data retrieval time. Like EFS, you can't directly connect it to CloudFront, unlike Amazon S3.

References:

<https://aws.amazon.com/getting-started/tutorials/deliver-content-faster/>

<https://aws.amazon.com/cloudfront/>

<https://aws.amazon.com/blogs/networking-and-content-delivery/amazon-s3-amazon-cloudfront-a-match-made-in-the-cloud/>

Check out these Amazon S3 and CloudFront Cheat Sheets:

<https://tutorialsdojo.com/amazon-s3/>

<https://tutorialsdojo.com/amazon-cloudfront/>

10. Question

Category: CDA – Development with AWS Services

A tech company has a real-time traffic monitoring system which uses Amazon Kinesis Data Stream to collect data and a group of EC2 instances that consume and process the data stream. Your development team is responsible for adjusting the number of shards in the data stream to adapt to changes in the rate of data flow.

Which of the following are correct regarding Kinesis resharding which your team should consider in managing the application? (Select TWO.)

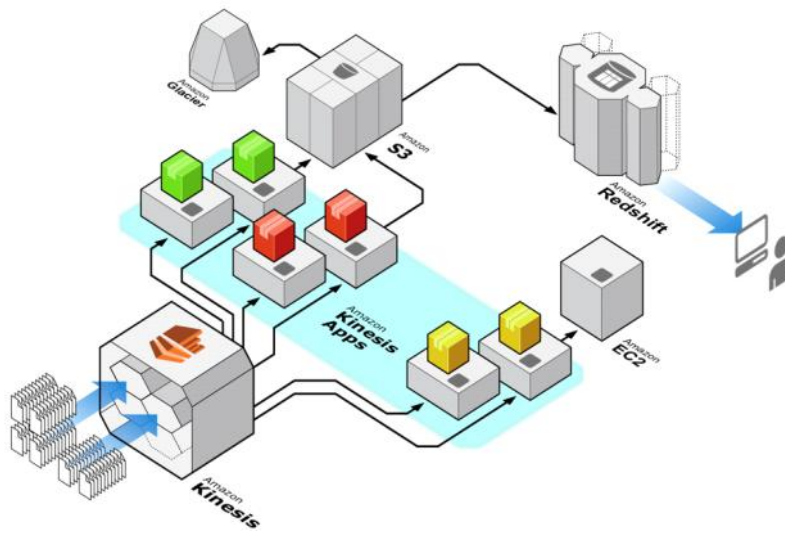
- You can decrease the stream's capacity by merging shards.
- The data records that are flowing to the parent shards will be lost when you reshard.
- You have to split the cold shards to decrease the capacity of the stream.
- You can increase the stream's capacity by splitting shards.
- You have to merge the hot shards to increase the capacity of the stream.

Incorrect

Amazon Kinesis Data Streams is a massively scalable, highly durable data ingestion and processing service optimized for streaming data. You can configure hundreds of thousands of data producers to continuously put data into a Kinesis data stream. Data will be available within milliseconds to your Amazon Kinesis applications, and those applications will receive data records in the order they were generated.

The purpose of resharding in Amazon Kinesis Data Streams is to enable your stream to adapt to changes in the rate of data flow. You split shards to increase the capacity (and cost) of your stream. You merge shards to reduce the cost (and capacity) of your stream.

One approach to resharding could be to split every shard in the stream—which would double the stream's capacity. However, this might provide more additional capacity than you actually need and therefore create unnecessary costs.



You can also use metrics to determine which are your "hot" or "cold" shards, that is, shards that are receiving much more data, or much less data, than expected. You could then selectively split the hot shards to increase capacity for the hash keys that target those shards. Similarly, you could merge cold shards to make better use of their unused capacity.

You can obtain some performance data for your stream from the Amazon CloudWatch metrics that Kinesis Data Streams publishes. However, you can also collect some of your own metrics for your streams. One approach would be to log the hash key values generated by the partition keys for your data records. Recall that you specify the partition key at the time that you add the record to the stream.

Hence, the correct statements regarding Kinesis resharding are:

- You can decrease the stream's capacity by merging shards
- You can increase the stream's capacity by splitting shards

The option that says: **you have to merge the hot shards to increase the capacity of the stream** is incorrect because a hot shard is the one that receives more data in the stream, which you should split rather than merge.

The option that says: **you have to split the cold shards to decrease the capacity of the stream** is incorrect because a cold shard is the one that receives fewer data in the stream, which you should merge rather than split. The option that says: **the data records that are flowing to the parent shards will be lost when you reshard** is incorrect because the data records are actually re-routed to flow to the child shards based on the hash key values that the data-record partition keys map to. Hence, this is incorrect because the records are not lost.

References:

<https://docs.aws.amazon.com/streams/latest/dev/kinesis-using-sdk-java-resharding-strategies.html>
<https://docs.aws.amazon.com/streams/latest/dev/kinesis-using-sdk-java-resharding.html>

Check out this Amazon Kinesis Cheat Sheet:

<https://tutorialsdojo.com/amazon-kinesis/>

Kinesis Scaling, Resharding and Parallel Processing:

<https://tutorialsdojo.com/kinesis-scaling-resharding-and-parallel-processing/>

11. Question

Category: CDA – Development with AWS Services

A company operates an e-commerce website on Amazon Elastic Container Service (ECS) behind an Application Load Balancer (ALB). They've set their ALB as an origin for an Amazon CloudFront distribution. Users interact with the website via a custom domain linked to the CloudFront distribution, all maintained within a public hosted zone in Amazon Route 53.

The company wants to display region-specific pricing tables to its users. For example, when a user from the UK visits the site, they should be redirected to <https://tutorialsdojo.com/uk/>, while users from the Philippines should view prices on <https://tutorialsdojo.com/ph/>.

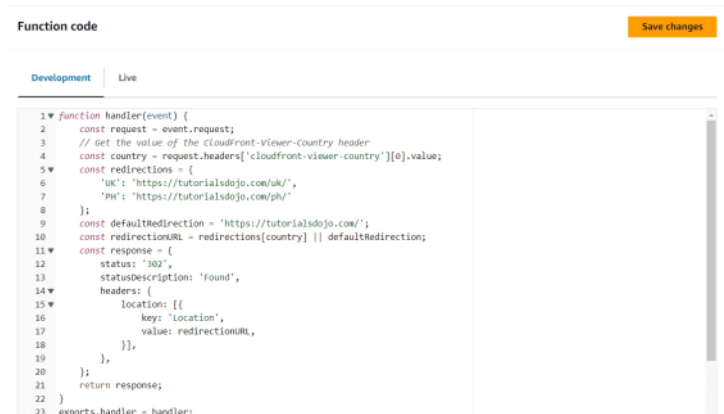
How can a developer incorporate this feature in the least amount of development overhead?

- Configure the Route 53 record to use the geolocation routing policy.
- Forward the CloudFront-Viewer-Address header to the web server running on the ECS cluster. Implement a custom logic that matches the header's value against a GeoIP database to determine user location. Based on the resolved location, redirect users to the appropriate region-specific URL.
- Use AWS Web Application Firewall (WAF's) geo-matching rule to identify the user country and attach it to the ALB. Configure ALB listener rules with path conditions to route traffic based on the identified country.
- Implement a CloudFront function that returns the appropriate URL based on the CloudFront-Viewer-Country. Configure the distribution to trigger the function on Viewer request events.

Incorrect

You can configure CloudFront to add specific HTTP headers to the requests that CloudFront receives from viewers and forwards to your origin or [edge function](#). The values of these HTTP headers are based on the characteristics of the viewer or the viewer request. The headers provide information about the viewer's device type, IP address, geographic location, request protocol (HTTP or HTTPS), HTTP version, TLS connection details, and JA3 fingerprint. With these headers, your origin or your edge function can receive information about the viewer without the need for you to write your own code to determine this information. If your origin returns different responses based on the information in these headers, you can include them in the *cache key* so that CloudFront caches the responses separately. For example, your origin might respond with content in a specific language based on the country that the viewer is in or with content tailored to a specific device type. Your origin might also write these headers to log files, which you can use to determine information about where your viewers are, which device types they're on, and more. In the given scenario, when a user initiates a request to the website, a CloudFront function can be triggered to inspect the CloudFront-Viewer-Country header, which pinpoints the originating country of a user. CloudFront functions are lightweight Javascript code that you can use to manipulate web requests at the CloudFront edge locations. By using a CloudFront function triggered on "Viewer request" events, you can assess and act upon incoming requests even before they reach the origin or CloudFront retrieves a cached response.

Here's a representation using a CloudFront function code snippet:



Hence, the correct answer is: **Implement a CloudFront function that returns the appropriate URL based on the CloudFront-Viewer-Country. Configure the distribution to trigger the function on Viewer request events.**

The option that says: **Forward the CloudFront-Viewer-Address header to the web server running on the ECS cluster. Implement a custom logic that matches the header's value against a GeoIP database to determine user location. Based on the resolved location, redirect users to the appropriate region-specific URL** is incorrect. While this approach is technically valid, it's more complex since you have to handle the IP-to-location translation on the backend and maintain an up-to-date GeoIP database.

The option that says: **Configure the Route 53 record to use the geolocation routing policy** is incorrect. Route 53 geolocation routing is primarily used for directing traffic to specific resources based on user location for performance or regulatory reasons, not for content personalization based on geolocation.

The option that says: **Use AWS Web Application Firewall (WAF's) geo-matching rule to identify the user country and attach it to the ALB. Configure ALB listener rules with path conditions to route traffic based on the identified country** is incorrect. AWS WAF's geo-matching rule identifies a user's country based on their IP. However, it's primarily designed to allow or block access, not for redirection. Even if used in conjunction with ALB, the ALB's listener rules can't inherently make routing decisions based on a viewer's geolocation.

References:

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/adding-cloudfront-headers.html>
<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/cloudfront-functions.html>

Check out this Amazon CloudFront Cheat Sheet:

<https://tutorialsdojo.com/amazon-cloudfront/>

12. Question

Category: CDA – Development with AWS Services

A developer has recently completed a new version of a serverless application that is ready to be deployed using AWS SAM. There is a requirement that the traffic should shift from the previous Lambda function to the new version in the shortest time possible, but you still don't want to shift traffic all-at-once immediately.

Which deployment configuration is the MOST suitable one to use in this scenario?

- `CodeDeployDefault.LambdaCanary10Percent5Minutes`
- `CodeDeployDefault.HalfAtATime`
- `CodeDeployDefault.LambdaLinear10PercentEvery1Minute`
- `CodeDeployDefault.LambdaLinear10PercentEvery2Minutes`

Incorrect

If you use AWS SAM to create your serverless application, it comes built-in with CodeDeploy to help ensure safe Lambda deployments. There are various deployment preference types that you can choose from.

For example:

If you choose `Canary10Percent10Minutes` then 10 percent of your customer traffic is immediately shifted to your new version. After 10 minutes, all traffic is shifted to the new version.

However, if your pre-hook/post-hook tests fail, or if a CloudWatch alarm is triggered, CodeDeploy rolls back your deployment. The following table outlines other traffic-shifting options that are available:

- **Canary:** Traffic is shifted in two increments. You can choose from predefined canary options. The options specify the percentage of traffic that's shifted to your updated Lambda function version in the first increment, and the interval, in minutes, before the remaining traffic is shifted in the second increment.
- **Linear:** Traffic is shifted in equal increments with an equal number of minutes between each increment. You can choose from predefined linear options that specify the percentage of traffic that's shifted in each increment and the number of minutes between each increment.
- **All-at-once:** All traffic is shifted from the original Lambda function to the updated Lambda function version at once.

Hence, the `CodeDeployDefault.LambdaCanary10Percent5Minutes` option is correct because 10 percent of your customer traffic is immediately shifted to your new version. After 5 minutes, all traffic is shifted to the new version. This means that the entire deployment time will only take 5 minutes.

`CodeDeployDefault.HalfAtATime` is incorrect because this is only applicable for EC2/On-premises compute platform and not for Lambda.

`CodeDeployDefault.LambdaLinear10PercentEvery1Minute` is incorrect because it will add 10 percent of the traffic linearly to the new version every minute. Hence, all traffic will be shifted to the new version only after 10 minutes.

`CodeDeployDefault.LambdaLinear10PercentEvery2Minutes` is incorrect because it will add 10 percent of the traffic linearly to the new version every 2 minutes. Hence, all traffic will be shifted to the new version only after 20 minutes.

References:

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/automating-updates-to-serverless-apps.html>
<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless-deploying.html>

13. 13. Question

Category: CDA – Development with AWS Services

A media company seeks to protect its copyrighted images from unauthorized distribution. They want images uploaded to their Amazon S3 bucket to be automatically watermarked. A developer has already prepared the Lambda function for this image-processing job.

Which option must the developer configure to automatically invoke the function at each upload?

- Enable S3 Storage Lens to monitor the bucket and configure the Lambda function to be invoked whenever the metrics indicate a new object creation.
- Use S3 Object Lambda to process images on retrieval and apply watermarks dynamically before the images are served to users.
- Set up an Amazon S3 Event Notification to trigger the Lambda function when an `ObjectCreated:Put` event is detected in the bucket.
- Configure an S3 Lifecycle policy to transition images to the `INTELLIGENT_TIERING` storage class. Use S3 Inventory to generate a report of images that weren't watermarked and set up the Lambda function to process the report.

Incorrect

You can use the Amazon S3 Event Notifications feature to receive notifications when certain events happen in your S3 bucket. To enable notifications, add a notification configuration that identifies the events that you want Amazon S3 to publish. Make sure that it also identifies the destinations where you want Amazon S3 to send the notifications.

Create event notification Info

To enable notifications, you must first add a notification configuration that identifies the events you want Amazon S3 to publish and the destinations where you want Amazon S3 to send the notifications.

General configuration

Event name

Event name can contain up to 255 characters.

Prefix - *optional*

Limit the notifications to objects with key starting with specified characters.

Suffix - *optional*

Limit the notifications to objects with key ending with specified characters.

Event types

Specify at least one event for which you want to receive notifications. For each group, you can choose an event type for all events, or you can choose one or more individual events.

Object creation

☐ All object create events
s3:ObjectCreated:*

☒ Put
s3:ObjectCreated:Put

Amazon S3 can send event notification messages to the following destinations:

- Amazon SQS queue
- AWS Lambda function
- Amazon SNS topic
- Amazon EventBridge

In the given scenario, you can set up a notification for the `ObjectCreated:Put` event to immediately trigger a Lambda function when an object is uploaded to the S3 bucket.

Hence, the correct answer is: **Set up an Amazon S3 Event Notification to trigger the Lambda function when an `ObjectCreated:Put` event is detected in the bucket.**

The option that says: **Enable S3 Storage Lens to monitor the bucket and configure the Lambda function to be invoked whenever the metrics indicate a new object creation** is incorrect. S3 Storage Lens just provide visibility into storage usage and activity trends. It does not trigger actions or Lambda functions based on object operations.

The option that says: **Configure an S3 Lifecycle policy to transition images to the `INTELLIGENT_TIERING` storage class. Use S3 Inventory to generate a report of images that weren't watermarked and set up the Lambda function to process the report** is incorrect. S3 Lifecycle policies simply manage storage transitions and object expirations, not event-driven actions like invoking Lambda functions upon uploads. Moreover, S3 Inventory just provides object lists and their metadata, but it doesn't automatically invoke Lambda functions upon image uploads.

The option that says: **Use S3 Object Lambda to process images on retrieval and apply watermarks dynamically before the images are served to users** is incorrect because S3 Object Lambda is primarily designed to transform objects at retrieval, not at upload. While it can dynamically apply watermarks, it does so when the object is accessed, not as part of the upload process, which would lead to watermarking every time the image is retrieved rather than just once upon upload.

References:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html>
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/enable-event-notifications.html>

Check out this Amazon S3 Cheat Sheet:

<https://tutorialdojo.com/amazon-s3/>

Check out this blog about S3 Event Notification:

<https://tutorialdojo.com/amazon-s3-event-notifications/>

14. 14. Question

Category: CDA – Development with AWS Services

A developer is building a Docker application using Amazon ECS. The application requires containers to maintain long-lived connections and access specific ports on the host container instance to send or receive traffic using port mapping. Which component of ECS should the developer configure to properly implement this task?

- Service scheduler
- Container instance
- Task definition
- Container Agent

Incorrect

Port mappings allow containers to access ports on the host container instance to send or receive traffic. Port mappings are specified as part of the container definition, which can be configured in the task definition.

Amazon Elastic Container Service > Create new task definition

Amazon Elastic Container Service

- Clusters
- Namespaces
- Task definitions**
- Account settings

Install AWS Copilot

Amazon ECR

Repositories

AWS Batch

Documentation

Discover products

Subscriptions

Tell us what you think

Create new task definition

Task definition configuration

Task definition family Info
Specify a unique task definition family name.
console-sample-app-static
Up to 255 letters (uppercase and lowercase), numbers, hyphens, and underscores are allowed.

▼ Infrastructure requirements
Specify the infrastructure requirements for the task definition.

Launch type Info
Selection of the launch type will change task definition parameters.

☒ **AWS Fargate**
Serverless compute for containers.

☐ **Amazon EC2 instances**
Self-managed infrastructure using Amazon EC2 instances.

OS, Architecture, Network mode
Network mode is used for tasks and is dependent on the compute type selected.

Operating system/Architecture Info
Linux/X86_64

Network mode Info
awsvpc

Task size Info
Specify the amount of CPU and memory to reserve for your task.

CPU
1 vCPU

Memory
3 GB

For task definitions that use the awsvpc network mode, you should only specify the containerPort. The hostPort can be left blank or it must be the same value as the containerPort. Port mappings on Windows use the NetNAT gateway address rather than localhost. There is no loopback for port mappings on Windows, so you cannot access a container's mapped port from the host itself. Hence, the correct answer is: **Task Definition**.

The option that says: **Service scheduler** is incorrect because this only provides you the ability to run tasks manually (for batch jobs or single run tasks), with Amazon ECS placing tasks on your cluster for you. The service scheduler is ideally suited for long-running stateless services and applications but not for configuring port mappings.

The option that says: **Container instance** is incorrect because this is just an Amazon EC2 instance running the Amazon ECS container agent and registered into a cluster. When you run tasks with Amazon ECS, your tasks using the EC2 launch type are placed on your active container instances. However, you can't manually configure the port mappings directly on your container instances but through task definitions.

The option that says: **Container Agent** is incorrect because this only allows container instances to connect to your cluster. The Amazon ECS container agent is included in the Amazon ECS-optimized AMIs, but you can also install it on any Amazon EC2 instance that supports the Amazon ECS specification. As with the other incorrect options, you can't configure port mappings with this component.

References:
https://docs.aws.amazon.com/AmazonECS/latest/APIReference/API_PortMapping.html
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task_definition_parameters.html#container_definition_portmappings

Check out this Amazon ECS Cheat Sheet:
<https://tutorialsdojo.com/amazon-elastic-container-service-amazon-ecs/>

15. Question

Category: CDA – Development with AWS Services

A company has a website hosted in a multicontainer Docker environment in Elastic Beanstalk. There is a requirement to integrate the website with API Gateway, where it simply passes client-submitted method requests to the backend. It is important that the client and backend interact directly with no intervention from API Gateway after the API method is set up, except for known issues such as unsupported characters.

Which of the following integration types is the MOST suitable one to use to meet this requirement?

- AWS_PROXY
- AWS
- HTTP_PROXY
- HTTP

Incorrect

You can integrate an API method in your API Gateway with a custom HTTP endpoint of your application in two ways:

- HTTP proxy integration
- HTTP custom integration

In your API Gateway console, you can define the type of HTTP integration of your resource by checking, or not checking, the “Configure as proxy resource” checkbox. For example, this API Resource configuration is a type of *HTTP Proxy integration* since the appropriate checkbox is ticked:

> ProxyResourceForPetStore (micuyu3ltg) > Resources > / (7ryft47g0) > Create Show all hints ?

Resources

Actions

New Child Resource

Use this page to create a new child resource for your resource.

Configure as proxy resource ☒ ⓘ

Resource Name* proxy

Resource Path* / {proxy+}

You can add path parameters using brackets. For example, the resource path {username} represents a path parameter called 'username'. Configuring /{proxy+} as a proxy resource catches all requests to its sub-resources. For example, it works for a GET request to /foo. To handle requests to /, add a new ANY method on the / resource.

Enable API Gateway CORS ☒ ⓘ

* Required

Cancel Create Resource

With proxy integration, the setup is simple. You only need to set the HTTP method and the HTTP endpoint URI, according to the backend requirements, if you are not concerned with content encoding or caching. With custom integration, setup is more involved. In addition to the proxy integration setup steps, you need to specify how the incoming request data is mapped to the integration request and how the resulting integration response data is mapped to the method response. API Gateway supports the following endpoint ports: 80, 443 and 1024-65535.

Programmatically, you choose an integration type by setting the **type** property on the **Integration** resource. For the Lambda proxy integration, the value is **AWS_PROXY**. For the Lambda custom integration and all other AWS integrations, it is **AWS**. For the HTTP proxy integration and HTTP integration, the value is **HTTP_PROXY** and **HTTP**, respectively. For the mock integration, the **type** value is **MOCK**.

Since the integration type that is being described in the scenario fits the definition of an *HTTP proxy integration*, the correct answer in this scenario is to use the **HTTP_PROXY** integration type.

AWS is incorrect because this type is only used for Lambda custom integration. Take note that the scenario uses an application hosted in EC2 and not in Lambda.

AWS_PROXY is incorrect because this type is primarily used for Lambda proxy integration. The scenario didn't mention that it uses a serverless application or Lambda.

HTTP is incorrect because this type is only used for HTTP custom integration where you need to specify how the incoming request data is mapped to the integration request and how the resulting integration response data is mapped to the method response.

References:
<https://docs.aws.amazon.com/apigateway/latest/developerguide/setup-http-integrations.html>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-integration-types.html>

Check out this Amazon API Gateway Cheat Sheet:
<https://tutorialsdojo.com/amazon-api-gateway/>

16. Question

Category: CDA – Development with AWS Services

A company has assigned a developer to automate its department's patch management, data synchronization, and other recurring tasks. The developer needs a service to coordinate multiple AWS services into serverless workflows.

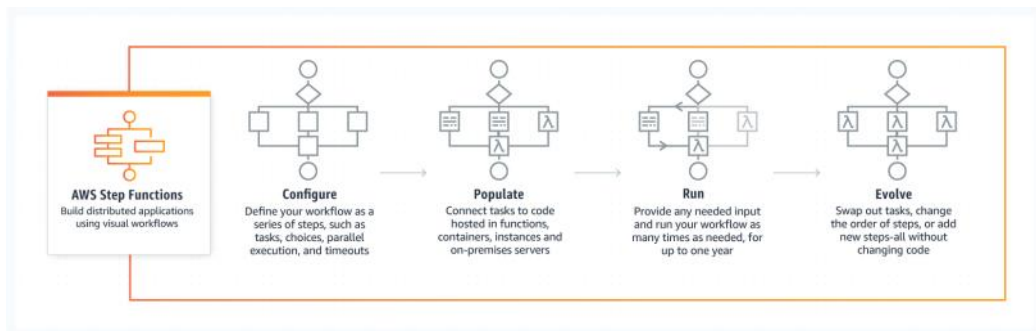
Which of the following is the MOST cost-effective service the developer should implement in this scenario?

- AWS Elastic Beanstalk

- AWS Lambda
- AWS Batch
- AWS Step Functions

Incorrect

AWS Step Functions provides serverless orchestration for modern applications. Orchestration centrally manages a workflow by breaking it into multiple steps, adding flow logic, and tracking the inputs and outputs between the steps. As your applications execute, Step Functions maintain the application state, tracking exactly which workflow step your application is in, and store an event log of data that is passed between application components. That means that if networks fail or components hang, your application can pick up right where it left off.



Application development is faster and more intuitive with Step Functions because you can define and manage the workflow of your application independently from its business logic. Making changes to one does not affect the other. You can easily update and modify workflows in one place, without having to struggle with managing, monitoring, and maintaining multiple point-to-point integrations. Step Functions frees your functions and containers from excess code, so your applications are faster to write, more resilient, and easier to maintain.

Hence, the correct answer is: **AWS Step Functions**.

AWS Elastic Beanstalk is incorrect because this service is for deploying and scaling web applications and services. However, it's not designed to coordinate multiple AWS services into serverless workflows.

Lambda is incorrect. Although it is typically used for serverless computing, it does not provide a direct way to coordinate multiple AWS services into serverless workflows.

AWS Batch is incorrect because it is primarily used to efficiently run hundreds of thousands of batch computing jobs in AWS.

References:

<https://aws.amazon.com/step-functions/features/>

<https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>

Check out this AWS Step Functions Cheat Sheet:

<https://tutorialsdojo.com/aws-step-functions/>

17. Question

Category: CDA – Development with AWS Services

The read and write operations to an Amazon DynamoDB table are throttled, causing errors in a stateful application that maintains user sessions. Despite checking Amazon CloudWatch metrics, the consumed capacity units have not exceeded the provisioned capacity. Upon further investigation, it is found that a “hot partition” is being accessed more frequently than others by downstream services.

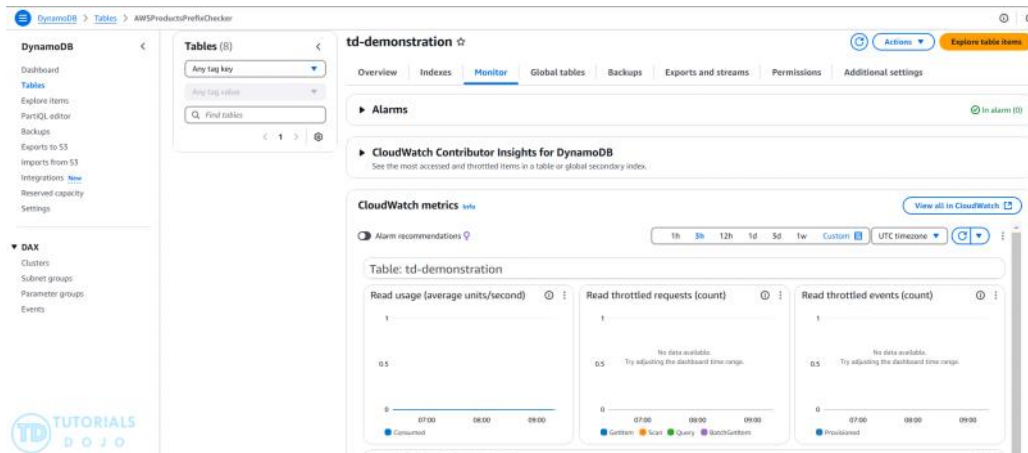
What should be done to resolve this issue with MINIMAL cost? (Select TWO.)

- Implement read sharding to distribute workloads evenly.
- Use DynamoDB Accelerator (DAX).
- Increase the amount of read or write capacity for your table.
- Implement error retries and exponential backoff.
- Refactor your application to distribute your read and write operations as evenly as possible across your table.

Incorrect

Partitions are usually throttled when they are accessed by your downstream applications much more frequently than other partitions (that is, a “hot” partition), or when workloads rely on short periods of time with high usage (a “burst” of read or write activity). This problem can be more pronounced in stateful applications, where maintaining session data or transactions may cause repeated access to the same partition. To avoid hot partitions and throttling, you must optimize your table and partition structure.

DynamoDB adaptive capacity automatically boosts throughput capacity to high-traffic partitions. However, each partition is still subject to the hard limit. This means that adaptive capacity can't solve larger issues with your table or partition design. To avoid hot partitions and throttling, optimize your table and partition structure.



To solve this issue, consider one or more of the following solutions:

- Increase the amount of read or write capacity for your table to anticipate short-term spikes or bursts in read or write operations. If you decide later you don't need the additional capacity, decrease it. Take note that Before deciding on how much to increase read or write capacity, consider the best practices in designing your partition keys.
- Implement error retries and exponential backoff. This technique improves reliability in stateful applications by allowing retries after progressively longer wait times between errors. AWS SDKs have built-in support for this logic.
- Distribute your read operations and write operations as evenly as possible across your table. A hot partition can degrade the overall performance, particularly for stateful applications that repeatedly access certain keys.
- Implement a caching solution, such as DynamoDB Accelerator (DAX) or Amazon ElastiCache. For stateful workloads with frequent reads to session or static data, caching can significantly reduce database access.

Hence, the correct answers are:

- Implement error retries and exponential backoff.
- Refactor your application to distribute your read and write operations as evenly as possible across your table.

The option that says: **Using a DynamoDB Accelerator (DAX)** is incorrect. Although it can reduce read load by caching frequently accessed items, it will simply incur additional costs to maintain the DAX cluster. Since the issue can be addressed by improving workload distribution without extra cost, DAX is not a cost-effective solution for this scenario.

The option that says: **Increasing the amount of read or write capacity for your table** is incorrect because, while it can alleviate throttling by temporarily handling higher traffic, it is an expensive solution that contradicts the minimal-cost requirement. Scaling read and write capacity (RCU and WCU) can lead to significant increases in operational costs, especially for stateful applications with continuous usage patterns.

The option that says: **Implementing read sharding to distribute workloads evenly** is incorrect because the issue primarily involves write operations in a stateful workload. Instead, write sharding should be used to better distribute writes across the partition key space. This can be achieved by appending either random or calculated suffixes to partition key values, which spreads the data more evenly across partitions.

References:

<https://aws.amazon.com/premiumsupport/knowledge-center/throttled-ddb/>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-sharding.html>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/best-practices.html>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

18. Question

Category: CDA – Development with AWS Services

A company has a development team that's heavily relying on AWS CodeBuild, and CodeDeploy. The management would like to further automate its CI/CD process. They requested a system that monitors the status of each code change, from the moment it's committed through to its deployment.

Which of the following AWS services will help you achieve this?

- AWS CodePipeline
- AWS Fault Injection Simulator
- AWS Elastic Beanstalk
- Amazon CodeGuru

Incorrect

AWS CodePipeline is a fully managed continuous delivery service that helps you automate your release pipelines for fast and reliable application and infrastructure updates. CodePipeline automates the build, test, and deploy phases of your release process every time there is a code change, based on the release model you define. This makes it a good choice for automating your CI/CD process and centrally monitoring application activity.

Moreover, AWS CodePipeline integrates with AWS CloudWatch, which provides a reliable, scalable, and flexible monitoring solution. You can create dashboards in CloudWatch to centrally monitor application activity and manage day-to-day development tasks.

The option that says: **AWS Fault Injection Simulator** is incorrect because this is just a managed service that is commonly used in chaos engineering, and not for application development. It enables you to perform fault injection experiments on your AWS workloads to improve the performance and resiliency of your applications.

The option that says: **Elastic Beanstalk** is incorrect because it is an orchestration service to quickly deploy and manage applications in AWS.

The option that says: **Amazon CodeGuru** is incorrect because this is simply a developer tool that provides intelligent recommendations to improve the quality of your codebase and for identifying an application's most "expensive" lines of code in terms of resource intensiveness, CPU performance, and code efficiency.

References:

<https://docs.aws.amazon.com/codepipeline/latest/userguide/detect-state-changes-cloudwatch-events.html>
<https://aws.amazon.com/codepipeline/>

Check out this AWS CodePipeline Cheat Sheet:

<https://tutorialsdojo.com/aws-codepipeline/>

19. Question

Category: CDA – Development with AWS Services

You are developing a high-traffic online stocks trading application, which will be hosted in an ECS Cluster and will be accessed by thousands of investors for intraday stocks trading. Each task of the cluster should be evenly placed across multiple Availability Zones to avoid any service disruptions.

Which of the following is the MOST suitable placementStrategy configuration that you should use in your task definition?

- "placementStrategy": [
 {
 "field": "attribute:ecs.availability-zone",
 "type": "spread"
 }
]
- "placementStrategy": [
 {
 "type": "random"
 }
]
- "placementStrategy": [
 {
 "field": "instanceId",
 "type": "spread"
 }
]
- "placementStrategy": [
 {
 "field": "memory",
 "type": "binpack"
 }
]

Incorrect

When a task that uses the EC2 launch type is launched, Amazon ECS must determine where to place the task based on the requirements specified in the task definition, such as CPU and memory. Similarly, when you scale down the task count, Amazon ECS must determine which tasks to terminate. You can apply task placement strategies and constraints to customize how Amazon ECS places and terminates tasks.

By default, tasks are randomly placed with RunTask or spread across Availability Zones with CreateService. Spread is typically used to achieve high availability by making sure that multiple copies of a task are scheduled across multiple instances based on attributes such as Availability Zones.

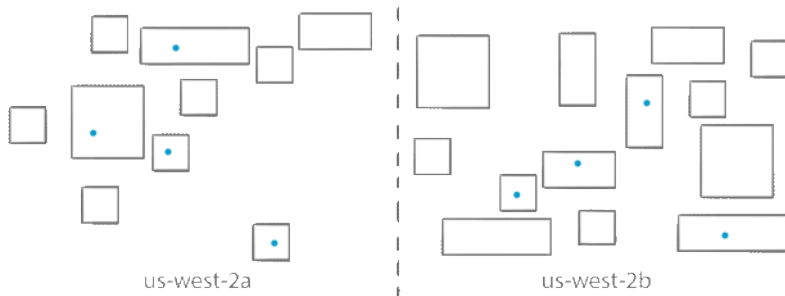
A **task placement strategy** is an algorithm for selecting instances for task placement or tasks for termination. Task placement strategies can be specified when either running a task or creating a new service.

Amazon ECS supports the following task placement strategies:

binpack – Place tasks based on the least available amount of CPU or memory. This minimizes the number of instances in use.

random – Place tasks randomly.

spread – Place tasks evenly based on the specified value. Accepted values are attribute key-value pairs, *instanceId*, or *host*.



The spread strategy, contrary to the binpack strategy, tries to put your tasks on as many different instances as possible. It is typically used to achieve high availability and mitigate risks, by making sure that you don't put all your task eggs in the same instance-baskets. Spread across Availability Zones, therefore, is the default placement strategy used for services.

When using the spread strategy, you must also indicate a *field* parameter. It is used to indicate the *bins* that you are considering. The accepted values are *instanceID*, *host*, or a custom attribute key:value pairs such as *attribute:ecs.availability-zone* to balance tasks across zones. There are several AWS attributes that start with the *ecs* prefix, but you can be creative and create your own attributes.

Hence, the task placement configuration which has a value of **"field": "attribute:ecs.availability-zone"**, **"type": "spread"** is correct, because this is using the appropriate strategy for task placement.

The configuration which has a value of **"field": "instanceid"**, **"type": "spread"** is incorrect because although it is using a spread task placement strategy, it distributes tasks evenly across all instances and not to Availability Zones.

The configuration which has a value of **"field": "memory"**, **"type": "binpack"** is incorrect because this is using a strategy that bin packs tasks based on memory. Take note that the scenario is asking for a configuration which will evenly place the tasks across multiple Availability Zones, and not based on memory.

The configuration which has a value of **"type": "random"** is incorrect because this will just place the tasks on instances randomly. You have to use the spread task placement strategy instead to meet the requirements of the scenario.

References:

<https://aws.amazon.com/blogs/compute/amazon-ecs-task-placement/>

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-placement.html>

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-placement-strategies.html>

Check out this Amazon ECS Cheat Sheet:

<https://tutorialsdojo.com/amazon-elastic-container-service-amazon-ecs/>

20. Question

Category: CDA – Development with AWS Services

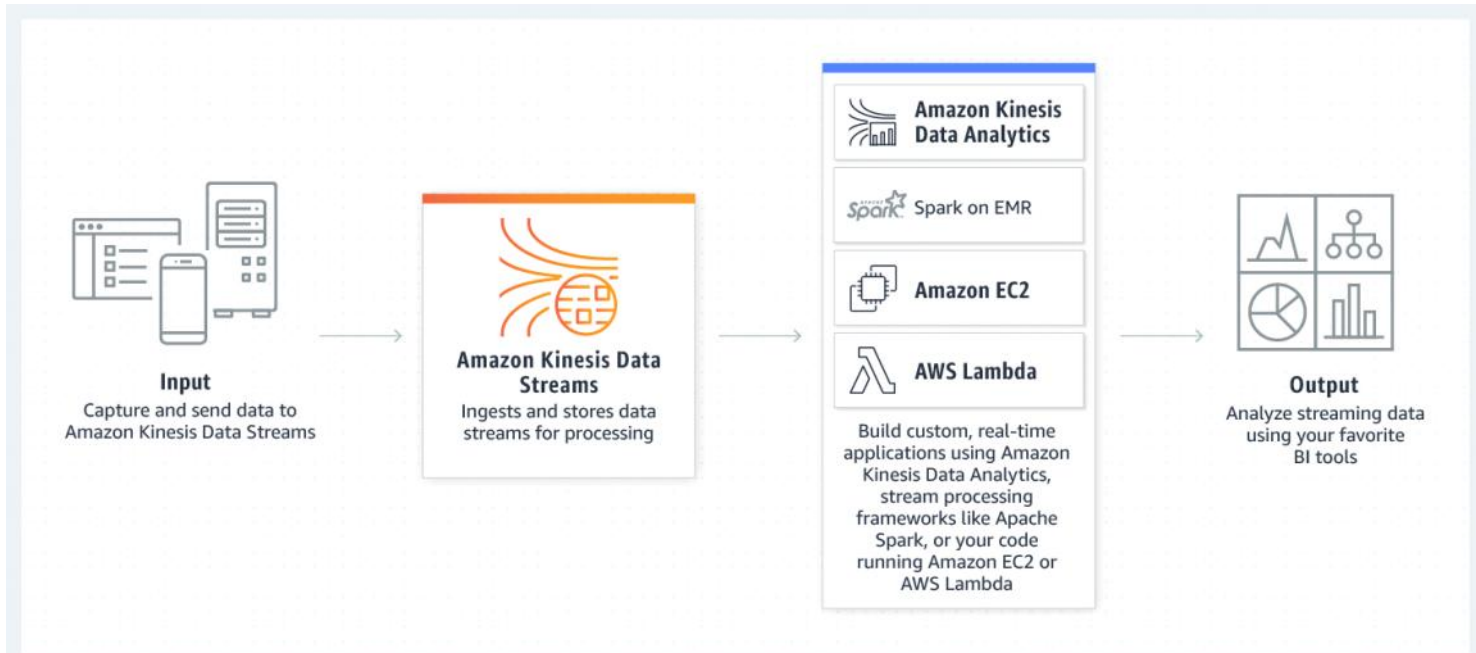
You are using an AWS Lambda function to process records in an Amazon Kinesis Data Streams stream which has 100 active shards. The Lambda function takes an average of 10 seconds to process the data and the stream is receiving 50 new items per second.

Which of the following statements are TRUE regarding this scenario?

- The Lambda function will throttle the incoming requests due to the excessive number of Kinesis shards.
- There will be at most 100 Lambda function invocations running concurrently.
- The Lambda function has 500 concurrent executions.
- The Kinesis shards must be merged to increase the data capacity of the stream as well as the concurrency execution of the Lambda function.

Incorrect

You can use an **AWS Lambda function** to process records in an Amazon Kinesis data stream. With Kinesis, you can collect data from many sources and process them with multiple consumers. Lambda supports standard data stream iterators and HTTP/2 stream consumers. Lambda reads records from the data stream and invokes your function synchronously with an event that contains stream records. Lambda reads records in batches and invokes your function to process records from the batch.



Concurrent executions refers to the number of executions of your function code that are happening at any given time. You can estimate the concurrent execution count, but the it will differ depending on whether or not your Lambda function is processing events from a poll-based event source.

For Lambda functions that process Kinesis or DynamoDB streams, the number of shards is the unit of concurrency. If your stream has 100 active shards, there will be at most 100 Lambda function invocations running concurrently. This is because Lambda processes each shard's events in sequence.

Hence, the correct answer in this scenario is that: **there will be at most 100 Lambda function invocations running concurrently.**

The option that says: **the Lambda function has 500 concurrent executions** is incorrect because the number of concurrent executions for poll-based event sources is different from push-based event sources. This number of concurrent executions would have been correct if the Lambda function is integrated with a push-based event source such as API Gateway or Amazon S3 Events. Remember that the Kinesis and Lambda integration is using a poll-based event source, which means that the number of shards is the unit of concurrency for the function.

The option that says: **the Lambda function will throttle the incoming requests due to the excessive number of Kinesis shards** is incorrect because, by default, AWS Lambda will automatically scale the function's concurrency execution in response to increased traffic, up to your concurrency limit. Moreover, having 100 shards is not excessive at all as long as there is a sufficient number of workers or consumers of the stream.

The option that says: **the Kinesis shards must be merged to increase the data capacity of the stream as well as the concurrency execution of the Lambda function** is incorrect because, in the first place, you have to split the shards in order to increase the data capacity of the stream and not merge them. Since the Lambda function is using a poll-based event mapping for Kinesis, the number of shards is the unit of concurrency for the function.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/with-kinesis.html>

<https://docs.aws.amazon.com/lambda/latest/dg/concurrent-executions.html>

<https://docs.aws.amazon.com/lambda/latest/dg/scaling.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>

21. Question

Category: CDA – Development with AWS Services

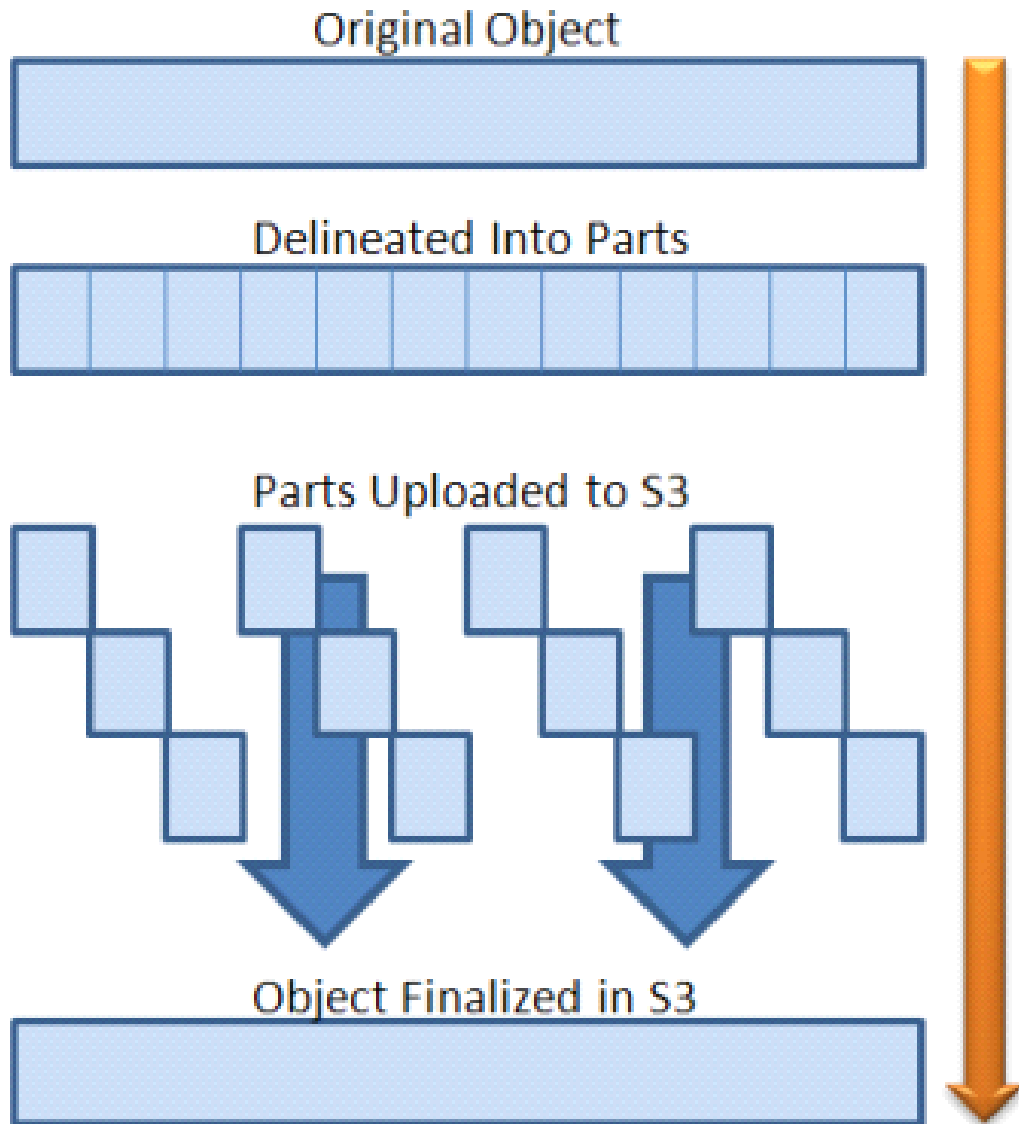
A web application is uploading large files, which are over 4 GB in size, to an Amazon S3 bucket called [data.tutorialsdojo.com](https://tutorialsdojo.com) every 30 minutes.

To minimize the time required for each upload, which of the following actions should be taken?

- Use the Multipart upload API.
- Use the PutItem API.
- Enable Transfer Acceleration in the bucket.
- Use the BatchWriteItem API.

Incorrect

Multipart upload allows you to upload a single object as a set of parts. Each part is a contiguous portion of the object's data. You can upload these object parts independently and in any order. If transmission of any part fails, you can retransmit that part without affecting other parts. After all parts of your object are uploaded, Amazon S3 assembles these parts and creates the object. In general, when your object size reaches 100 MB, you should consider using multipart uploads instead of uploading the object in a single operation.



Using multipart upload provides the following advantages:

- **Improved throughput** – You can upload parts in parallel to improve throughput.
- **Quick recovery from any network issues** – Smaller part size minimizes the impact of restarting a failed upload due to a network error.
- **Pause and resume object uploads** – You can upload object parts over time. Once you initiate a multipart upload, there is no expiry; you must explicitly complete or abort the multipart upload.
- **Begin an upload before you know the final object size** – You can upload an object as you are creating it.

Hence, the correct answer is: **Use the Multipart Upload API.**

The options that say: **Use the BatchWriteItem API** and **Use the PutItem API** are incorrect because these are primarily DynamoDB APIs and not S3.

The option that says: **Enable Transfer Acceleration in the bucket** is incorrect because although Transfer Acceleration will typically reduce the upload time to S3, the bucket in the scenario won't be able to turn on this feature. Take note that the name of the bucket used for Transfer Acceleration must be DNS-compliant and must not contain periods ("").

References:

<https://docs.aws.amazon.com/AmazonS3/latest/dev/uploadobjusingmpu.html>
<https://docs.aws.amazon.com/AmazonS3/latest/dev/transfer-acceleration.html>

Check out this Amazon S3 Cheat Sheet:

<https://tutorialsdojo.com/amazon-s3/>

22. Question

Category: CDA – Development with AWS Services

A company is re-architecting its legacy application to use AWS Lambda and DynamoDB. The table is provisioned to have 10 read capacity units, and each item has a size of 4 KB.

How many eventual and strong consistent read requests can the table handle per second?

- 10 strongly consistent reads and 20 eventually consistent reads per second
- 20 strongly consistent reads and 10 eventually consistent reads per second
- 10 strongly consistent reads and 10 eventually consistent reads per second
- 5 strongly consistent reads and 20 eventually consistent reads per second

Incorrect

One *read request unit* represents one strongly consistent read request, or two eventually consistent read requests, for an item up to 4 KB in size. Transactional read requests require 2 read request units to perform one read for items up to 4 KB.

If you need to read an item that is larger than 4 KB, DynamoDB needs additional read request units. The total number of read request units required depends on the item size, and whether you want an eventually consistent or strongly consistent read.

Capacity calculator

Avg. item size	2	KB
Item read/sec	20	Eventually consistent ▾
Item write/sec	10	Standard ▾
Read capacity	10	
Write capacity	20	
Estimated cost per table/index	\$10.64 / month	

Cancel

Update

For example, suppose that you create a table with 10 provisioned read capacity units. This allows you to perform 10 strongly consistent reads per second, or 20 eventually consistent reads per second, for items up to 4 KB. Reading an item larger than 4 KB consumes more read capacity units. For example, a strongly consistent read of an item that is 8 KB (4 KB × 2) consumes 2 read capacity units. An eventually consistent read on that same item consumes only 1 read capacity unit.

Item sizes for reads are rounded up to the next 4 KB multiple. For example, reading a 3,500-byte item consumes the same throughput as reading a 4 KB item.

To get the number of strong and eventual consistent read requests that your table can accommodate per second, you simply have to do the following steps:

Step #1 Multiply the value of the provisioned RCU by 4 KB

= 10 RCU × 4 KB

= 40

Step #2 To get the number of strong consistency requests, just divide the result of step 1 by 4 KB

= 40 / 4 KB

= 10 strongly consistent read requests

Step #3 To get the number of eventual consistency requests, just divide the result of step 1 by 2 KB

= 40 / 2 KB

= 20 eventually consistent read requests

Hence, the correct answer is that the table can handle **10 strongly consistent reads and 20 eventually consistent reads per second**.

10 strongly consistent reads and 10 eventually consistent reads per second is incorrect. Although the former value is correct, the latter one is not. Take note that one strongly consistent read request is equivalent to two eventually consistent read requests as these two consistency types are quite different from each other.

5 strongly consistent reads and 20 eventually consistent reads per second is incorrect. Although the latter value is correct, the former one is not. If the scenario says that it uses transaction read requests, then it is correct that it will provide 5 strongly consistent reads. However, it is explicitly mentioned to get both strong and eventual consistency.

20 strongly consistent reads and 10 eventually consistent reads per second is incorrect because it should be the other way around. The table can provide 10 strongly consistent reads and 20 eventually consistent reads per second.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ProvisionedThroughput.html#ItemSizeCalculations.Reads>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsofdojo.com/amazon-dynamodb/>

Calculating the Required Read and Write Capacity Unit for your DynamoDB Table:

<https://tutorialsofdojo.com/calculating-the-required-read-and-write-capacity-unit-for-your-dynamodb-table/>

23. Question

Category: CDA – Development with AWS Services

A startup has recently launched their new mobile game and is gaining a lot of new users everyday. The founders plan to add a new feature which will enable cross-device syncing of user profile data across mobile devices to improve the user experience.

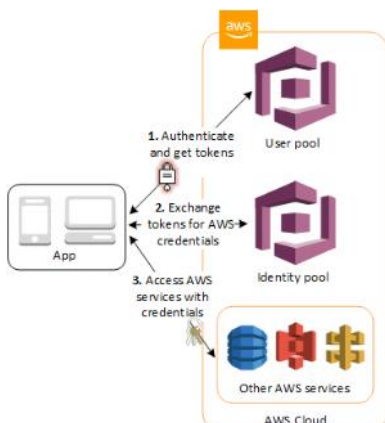
Which of the following services should they use to meet this requirement?

- Cognito Sync
- AWS Amplify
- Cognito User Pools
- Cognito Identity Pools

Incorrect

Amazon Cognito Sync is an AWS service and client library that enables cross-device syncing of application-related user data. You can use it to synchronize user profile data across mobile devices and the web without requiring your own backend. The client libraries cache data locally so your app can read and write data regardless of device connectivity status. When the device is online, you can synchronize data, and if you set up push sync, notify other devices immediately that an update is available.

Amazon Cognito lets you save end user data in datasets containing key-value pairs. This data is associated with an Amazon Cognito identity, so that it can be accessed across logins and devices. To sync this data between the Amazon Cognito service and an end user's devices, invoke the synchronize method. Each dataset can have a maximum size of 1 MB. You can associate up to 20 datasets with an identity.



The Amazon Cognito Sync client creates a local cache for the identity data. Your app talks to this local cache when it reads and writes keys. This guarantees that all of your changes made on the device are immediately available on the device, even when you are offline. When the synchronize method is called, changes from the service are pulled to the device, and any local changes are pushed to the service. At this point the changes are available to other devices to synchronize.

Amazon Cognito automatically tracks the association between identity and devices. Using the push synchronization, or push sync, feature, you can ensure that every instance of a given identity is notified when identity data changes.

Push sync ensures that, whenever the sync store data changes for a particular identity, all devices associated with that identity receive a silent push notification informing them of the change.

Hence, the correct answer is to use **Cognito Sync**.

Cognito User Pools is incorrect because this is just a user directory that allows your users to sign in to your web or mobile app through Amazon Cognito.

Cognito Identity Pools is incorrect because this simply enables you to create unique identities for your users and federate them with identity providers where you can obtain temporary, limited-privilege AWS credentials to access other AWS services.

AWS Amplify is incorrect because this just makes it easy for you to create, configure, and implement scalable mobile and web apps powered by AWS. It does not have the ability to synchronize user profile data across mobile devices.

References:

<https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-sync.html>

<https://docs.aws.amazon.com/cognito/latest/developerguide/push-sync.html>

24. 24. Question

Category: CDA – Development with AWS Services

A company is using OpenAPI, which is also known as Swagger, for the API specifications of their REST web services that are hosted on their on-premises data center. They want to migrate their system to AWS using Lambda and API Gateway. In line with this, you are instructed to create a new API and populate it with the resources and methods from their Swagger definition.

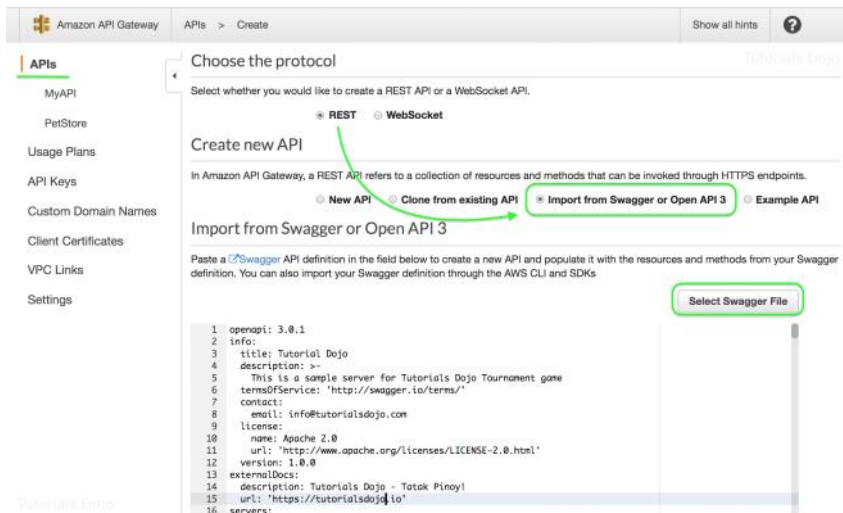
Which of the following is the EASIEST way to accomplish this task?

- Use CodeDeploy to migrate and deploy the company's web services to API Gateway.
- Use AWS SAM to migrate and deploy the company's web services to API Gateway.
- Import their Swagger or OpenAPI definitions to API Gateway using the AWS Console.
- Create models and templates for request and response mappings based on the company's API definitions.

Incorrect

You can use the API Gateway Import API feature to import a REST API from an external definition file into API Gateway. Currently, the Import API feature supports OpenAPI v2.0 and OpenAPI v3.0 definition files. You can update an API by overwriting it with a new definition or merge a definition with an existing API. You specify the options using a mode query parameter in the request URL.

You can paste a [Swagger](#) API definition in the AWS Console to create a new API and populate it with the resources and methods from your Swagger or OpenAPI definition, just as shown below:



You can also import your Swagger definition through the AWS CLI and SDKs.

Hence, the correct answer in this scenario is to **import their Swagger or OpenAPI definitions to API Gateway using the AWS Console**.

Using CodeDeploy to migrate and deploy the company's web services to API Gateway is incorrect because using CodeDeploy alone is not enough to deploy new custom APIs. This is mainly used in conjunction with AWS SAM where you can add deployment preferences to manage the way traffic is shifted during an AWS Lambda application deployment.

Using AWS SAM to migrate and deploy the company's web services to API Gateway is incorrect. Although using AWS SAM is the preferred way to deploy your serverless application, it is not the easiest way to import the Swagger API definitions file. As mentioned above, you can simply import Swagger or OpenAPI files directly to AWS.

Creating models and templates for request and response mappings based on the company's API definitions is incorrect because this is primarily done for API Gateway integration to other services and not for importing API definitions file.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-import-api.html>

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-create-api-from-example.html>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdjoio.com/amazon-api-gateway/>

25. 25. Question

Category: CDA – Development with AWS Services

A developer is planning to add a global secondary index in a DynamoDB table. This will allow the application to query a specific index that can span all of the data in the base table, across all partitions.

Which of the following should the developer consider when using this type of index? (Select TWO.)

- Queries or scans on this index consume read capacity units from the base table.
- Queries or scans on this index consume capacity units from the index, not from the base table.
- Queries on this index support eventual consistency only.
- When you query this index, you can choose either eventual consistency or strong consistency.
- For each partition key value, the total size of all indexed items must be 10 GB or less.

Incorrect

A global secondary index is an index with a partition key and a sort key that can be different from those on the base table. A global secondary index is considered "global" because queries on the index can span all of the data in the base table, across all partitions.

To create a table with one or more global secondary indexes, use the CreateTable operation with the GlobalSecondaryIndexes parameter. For maximum query flexibility, you can create up to 20 global secondary indexes (default limit) per table. You must specify one attribute to act as the index partition key; you can optionally specify another attribute for the index sort key. It is not necessary for either of these key attributes to be the same as a key attribute in the table. Global secondary indexes inherit the read/write capacity mode from the base table.

Characteristic	Global Secondary Index	Local Secondary Index
Key Schema	The primary key of a global secondary index can be either simple (partition key) or composite (partition key and sort key).	The primary key of a local secondary index must be composite (partition key and sort key).
Key Attributes	The index partition key and sort key (if present) can be any base table attributes of type string, number, or binary.	The partition key of the index is the same attribute as the partition key of the base table. The sort key can be any base table attribute of type string, number, or binary.
Size Restrictions Per Partition Key Value	There are no size restrictions for global secondary indexes.	For each partition key value, the total size of all indexed items must be 10 GB or less.
Online Index Operations	Global secondary indexes can be created at the same time that you create a table. You can also add a new global secondary index to an existing table, or delete an existing global secondary index. For more information, see Managing Global Secondary Indexes .	Local secondary indexes are created at the same time that you create a table. You cannot add a local secondary index to an existing table, nor can you delete any local secondary indexes that currently exist.
Queries and Partitions	A global secondary index lets you query over the entire table, across all partitions.	A local secondary index lets you query over a single partition, as specified by the partition key value in the query.
Read Consistency	Queries on global secondary indexes support eventual consistency only.	When you query a local secondary index, you can choose either eventual consistency or strong consistency.
Provisioned Throughput Consumption	Every global secondary index has its own provisioned throughput settings for read and write activity. Queries or scans on a global secondary index consume capacity units from the index, not from the base table. The same holds true for global secondary index updates due to table writes.	Queries or scans on a local secondary index consume read capacity units from the base table. When you write to a table, its local secondary indexes are also updated; these updates consume write capacity units from the base table.
Projected Attributes	With global secondary index queries or scans, you can only request the attributes that are projected into the index. DynamoDB will not fetch any attributes from the table.	If you query or scan a local secondary index, you can request attributes that are not projected in to the index. DynamoDB will automatically fetch those attributes from the table.

As shown in the above table, the following are the things that the developer should consider when using a global secondary index:

- Queries or scans on this index consume capacity units from the index, not from the base table.
- Queries on this index support eventual consistency only.

The following options are incorrect because these are about local secondary indexes:

- When you query this index, you can choose either eventual consistency or strong consistency.
- Queries or scans on this index consume read capacity units from the base table
- For each partition key value, the total size of all indexed items must be 10 GB or less.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/SecondaryIndexes.html>
<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

DynamoDB Scan vs Query:

<https://tutorialsdojo.com/dynamodb-scan-vs-query/>

Comparison of AWS Services Cheat Sheets:

<https://tutorialsdojo.com/comparison-of-aws-services/>

26. Question

Category: CDA – Development with AWS Services

A developer is creating a script using AWS CLI to retrieve a list of objects in an S3 bucket. However, the script is timing out if the bucket has tens of thousands of objects.

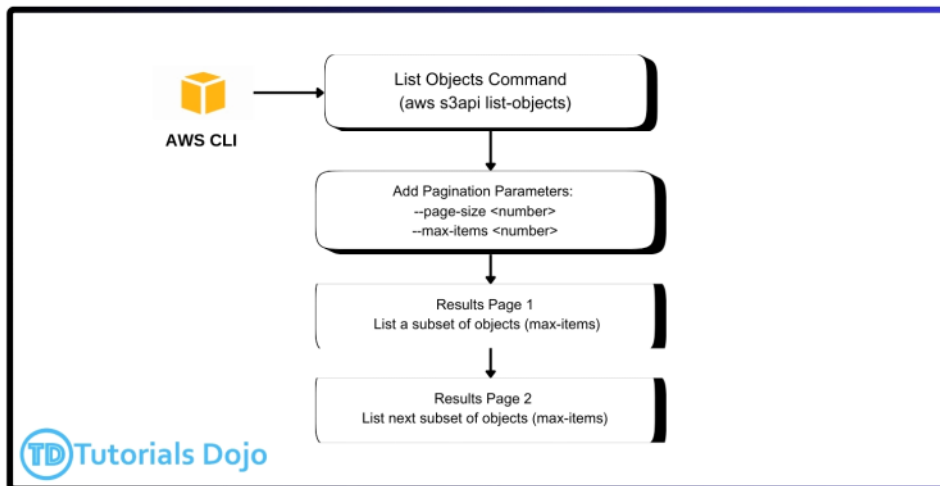
Which solution would most likely rectify the issue?

- Increase the AWS CLI timeout value
- Enable Amazon S3 Transfer Acceleration
- Enable CORS
- Apply the pagination parameters in the AWS CLI command

Incorrect

For commands that possibly return a long list of items, the AWS CLI provides parameters allowing you to limit the number of items included in the output when the AWS CLI queries a service's API.

By default, the AWS CLI retrieves all accessible items with a page size of 1,000. If you need help running list commands on a large number of resources, the default page size of 1000 may be too large. This can cause calls to AWS services to exceed the maximum allowed time, resulting in a "timed out" error.



One of the pagination options you can use is the `--page-size` option. This option tells the AWS CLI to request a smaller number of items from each call to the AWS service.

aws s3api list-objects --bucket tdbucket --page-size 100

The CLI still retrieves the entire list, but it makes a greater number of service API calls in the background and retrieves fewer items with each request. This increases the probability that individual calls will succeed without the use of a timeout.

Hence, the correct answer is: **Apply the pagination parameters in the AWS CLI command.**

The option that says: **Increase the AWS CLI timeout value** is incorrect. Increasing CLI parameters like `--cli-connect-timeout` or `--cli-read-timeout` would only prolong the process and increase susceptibility to timeouts due to network latency. On the other hand, pagination would handle large data sets by retrieving objects in manageable chunks, aligning with S3's response limits and preventing timeouts.

The option that says: **Enabling Amazon S3 Transfer Acceleration** is incorrect because this is only a bucket-level feature that enables faster data transfers to and from Amazon S3. Although this will improve the retrieval times of your objects, this feature will still not paginate the result, which may still cause time-out errors.

The option that says: **Enabling CORS** is incorrect because the Cross-origin resource sharing (CORS) is simply that allows client web applications that are loaded in one domain to communicate with resources in a different domain. This is not useful in paginating the results from an AWS CLI call.

References:

<https://docs.aws.amazon.com/cli/latest/userguide/cli-usage-pagination.html>
<https://docs.aws.amazon.com/cli/latest/reference/s3api/list-objects.html>

Check out this AWS CloudShell Cheat Sheet:

<https://tutorialsdojo.com/aws-cloudshell/>

27. 27. Question

Category: CDA – Development with AWS Services

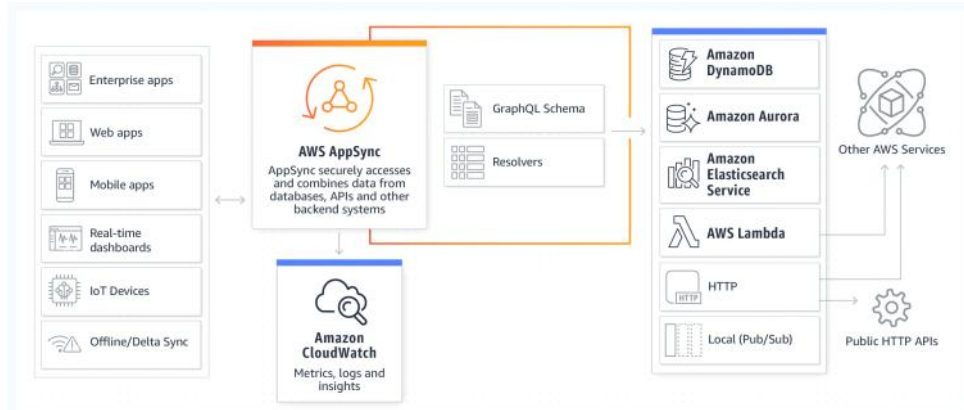
You are developing an online game where the app preferences and game state of the player must be synchronized across devices. It should also allow multiple users to synchronize and collaborate shared data in real time. Which of the following is the MOST appropriate solution that you should implement in this scenario?

- Integrate Amazon Pinpoint to your mobile app.
- Integrate Amazon Cognito Sync to your mobile app.
- Integrate AWS AppSync to your mobile app.
- Integrate AWS Amplify to your mobile app.

Incorrect

AWS AppSync simplifies application development by letting you create a flexible API to securely access, manipulate, and combine data from one or more data sources. AppSync is a managed service that uses GraphQL to make it easy for applications to get exactly the data they need.

With AppSync, you can build scalable applications, including those requiring real-time updates, on a range of data sources such as NoSQL data stores, relational databases, HTTP APIs, and your custom data sources with AWS Lambda. For mobile and web apps, AppSync additionally provides local data access when devices go offline, and data synchronization with customizable conflict resolution, when they are back online.



AWS AppSync is quite similar with Amazon Cognito Sync which is also a service for synchronizing application data across devices. It enables user data like app preferences or game state to be synchronized as well however, the key difference is that, it also extends these capabilities by allowing multiple users to synchronize and collaborate in real time on shared data.

Hence, the correct answer is to **integrate AWS AppSync to your mobile app**.

Integrating AWS Amplify to your mobile app is incorrect because this service just makes it easy to create, configure, and implement scalable mobile and web apps powered by AWS. This is primarily used to automate the application release process of both your frontend and backend allowing you to deliver features faster, and not for synchronizing application data across devices.

Integrating Amazon Cognito Sync to your mobile app is incorrect. Although this service can also be used in synchronizing application data across devices, it does not allow multiple users to synchronize and collaborate in real-time on shared data, unlike AWS AppSync.

Integrating Amazon Pinpoint to your mobile app is incorrect because this service simply allows you to engage with your customers across multiple messaging channels. This is primarily used to send push notifications, emails, SMS text messages, and voice messages.

References:

<https://aws.amazon.com/appsync/>

<https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-sync.html>

28. 28. Question

Category: CDA – Development with AWS Services

A mobile game is using a DynamoDB table named GameScore that keeps track of users and scores. Each item in the table is identified by a partition key (UserId) and a sort key (GameTitle). The diagram below shows how the items in the table are organized:

GameScores

UserId	GameTitle	TopScore	TopScoreDateTime	Wins	Losses
"101"	"Galaxy Invaders"	5842	"2015-09-15:17:24:31"	21	72
"101"	"Meteor Blasters"	1000	"2015-10-22:23:18:01"	12	3
"101"	"Starship X"	24	"2015-08-31:13:14:21"	4	9
"102"	"Alien Adventure"	192	"2015-07-12:11:07:56"	32	192
"102"	"Galaxy Invaders"	0	"2015-09-18:07:33:42"	0	5
"103"	"Attack Ships"	3	"2015-10-19:01:13:24"	1	8
"103"	"Galaxy Invaders"	2317	"2015-09-11:06:53:00"	40	3
"103"	"Meteor Blasters"	723	"2015-10-19:01:13:24"	22	12
"103"	"Starship X"	42	"2015-07-11:06:53:00"	4	19

A developer wants to write a leaderboard application to display the top scores for each game.

How can the developer meet the requirement in the MOST efficient manner?

- Create a global secondary index. Assign the GameTitle attribute as the partition key and the TopScore attribute as the sort key.
- Create a local secondary index. Assign the TopScore attribute as the partition key and the GameTitle attribute as the sort key.
- Use the Scan operation and filter the results based on a GameTitle value.
- Create a local secondary index. Assign the GameTitle attribute as the partition key and the TopScore attribute as the sort key.

Incorrect

Amazon DynamoDB provides fast access to items in a table by specifying primary key values. However, many applications might benefit from having one or more secondary (or alternate) keys available, to allow efficient access to data with attributes other than the primary key. To address this, you can create one or more secondary indexes on a table, and issue *Query* or *Scan* requests against these indexes.

A **secondary index** is a data structure that contains a subset of attributes from a table, along with an alternate key to support *Query* operations. You can retrieve data from the index using a *Query*, in much the same way as you use *Query* with a table. A table can have multiple secondary indexes, which gives your applications access to many different query patterns.

DynamoDB supports two types of secondary indexes:

Global secondary index — an index with a partition key and a sort key that can be different from those on the base table. A global secondary index is considered “global” because queries on the index can span all of the data in the base table, across all partitions.

Local secondary index — an index that has the same partition key as the base table, but a different sort key. A local secondary index is “local” in the sense that every partition of a local secondary index is scoped to a base table partition that has the same partition key value.

To speed up queries on non-key attributes, you can create a global secondary index. A global secondary index contains a selection of attributes from the base table, but they are organized by a primary key that is different from that of the table. The index key does not need to have any of the key attributes from the table; it doesn’t even need to have the same key schema as a table.

In this scenario, you could create a global secondary index named *GameTitleIndex*, with a partition key of *GameTitle* and a sort key of *TopScore*. Since the base table’s primary key attributes are always projected into an index, the *UserId* attribute is also present. The following diagram shows what *GameTitleIndex* index would look like:

GameTitleIndex

GameTitle	TopScore	UserId
"Alien Adventure"	192	"102"
"Attack Ships"	3	"103"
"Galaxy Invaders"	0	"102"
"Galaxy Invaders"	2317	"103"
"Galaxy Invaders"	5842	"101"
"Meteor Blasters"	723	"103"
"Meteor Blasters"	1000	"101"
"Starship X"	24	"101"
"Starship X"	42	"103"

Hence, the correct answer in this scenario is to: **Create a global secondary index. Assign the GameTitle attribute as the partition key and the TopScore attribute as the sort key.**

Create a local secondary index. Assign the TopScore attribute as the partition key and the GameTitle attribute as the sort key is incorrect. You can't add a local secondary index to an existing table. Moreover, even if it's possible, making a query that returns the top scores for each game is impossible with the TopScore attribute as the partition key. When you issue a query, you must also specify a partition key. In this case, if you run a query with a partition key value of 500, the results might return different games with a score of 500 from various users. It does not tell if it's the highest score in that game.

Create a local secondary index. Assign the GameTitle attribute as the partition key and the TopScore attribute as the sort key is incorrect because you can't add this index to an already existing table. Additionally, a local secondary index has the same partition key as the base table, but has a different sort key. It is "local" in the sense that every partition of a local secondary index is scoped to a base table partition that has the same partition key value. **Use the Scan operation and filter the results based on a GameTitle value** is incorrect. Technically, this also works but it is less efficient and slower compared to querying on secondary indexes. The Scan operation reads every item in a table. As the table grows, the slower the Scan operation would become.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/SecondaryIndexes.html>
<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

DynamoDB Scan vs Query:

<https://tutorialsdojo.com/dynamodb-scan-vs-query/>

29. Question

Category: CDA – Development with AWS Services

You recently deployed an application to a newly created AWS account, which uses two identical Lambda functions to process ad-hoc requests. The first function processes incoming requests efficiently but the second one has a longer processing time even though both of the functions have exactly the same code. Based on your monitoring, the **Throttles** metric of the second function is greater than the first one in Amazon CloudWatch.

Which of the following are possible solutions that you can implement to fix this issue? (Select TWO.)

- Set the concurrency execution limit of both functions to 500.
- Decrease the concurrency execution limit of the first function.
- Set the concurrency execution limit of the second function to 0.
- Set the concurrency execution limit of both functions to 450.
- Configure the second function to use an unreserved account concurrency.

Incorrect

The unit of scale for AWS Lambda is a concurrent execution. However, scaling indefinitely is not desirable in all scenarios. For example, you may want to control your concurrency for cost reasons or to regulate how long it takes you to process a batch of events, or to simply match it with a downstream resource. To assist with this, Lambda provides a concurrent execution limit control at both the account level and the function level.

The *concurrent executions* refers to the number of executions of your function code that are happening at any given time. You can estimate the concurrent execution count, but the concurrent execution count will differ depending on whether or not your Lambda function is processing events from a poll-based event source.

Concurrency

Unreserved account concurrency 100

- ☐ Use unreserved account concurrency
- ☒ Reserve concurrency 900

The unreserved account concurrency can't go below 100.

Tutorials Dojo

If you create a Lambda function to process events from event sources that aren't poll-based (for example, Lambda can process every event from other sources, like Amazon S3 or API Gateway), each published event is a unit of work, in parallel, up to your account limits. Therefore, the number of invocations these event sources make influences the concurrency.

If you set the concurrent execution limit for a function, the value is deducted from the unreserved concurrency pool. For example, if your account's concurrent execution limit is 1000 and you have 10 functions, you can specify a limit on one function at 200 and another function at 100. The remaining 700 will be shared among the other 8 functions.

AWS Lambda will keep the unreserved concurrency pool at a minimum of 100 concurrent executions, so that functions that do not have specific limits set can still process requests. So, in practice, if your total account limit is 1000, you are limited to allocating 900 to individual functions.

By default, an AWS account's concurrent execution limit is 1000 which will be shared by all Lambda functions. In this scenario, it is highly likely that the first function has more provisioned concurrency than the other one. It is possible that the concurrency execution limit of the first function is set to a significantly high value (e.g. 900) and the second function is set to use the unreserved account concurrency which may only contain the last 100 units out of the AWS account's concurrent execution limit of 1000.

Hence, the correct solutions in this scenario are:

- Set the concurrency execution limit of both functions to 450
- Decrease the concurrency execution limit of the first function.

Setting the concurrency execution limit of both functions to 500 is incorrect because by default, a newly created AWS account has a concurrent execution limit of only 1000. Take note that AWS Lambda will keep the unreserved concurrency pool at a minimum of 100 concurrent executions so that functions that do not have specific limits set can still process requests. Hence, you can only allocate a concurrent execution limit of 900 for a single Lambda function or 450 for two functions.

Configuring the second function to use an unreserved account concurrency is incorrect because this may possibly be the current setting of this function, which is why the requests are being throttled. The total number of concurrent execution limits that you allocated to all Lambda functions affect the value of the unreserved concurrency limit. Since the second function is being throttled, it is highly likely that it is already using an unreserved account concurrency, which only has a low value since the units were already exhausted by the first function. Take note that the unreserved concurrency pool has a minimum value of 100 concurrent executions.

Setting the concurrency execution limit of the second function to 0 is incorrect because this will throttle all future invocations of this function and will make the problem worse.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/concurrent-executions.html>
<https://docs.aws.amazon.com/lambda/latest/dg/scaling.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>

30. Question

Category: CDA – Development with AWS Services

A serverless application is using API Gateway with a non-proxy Lambda Integration. A developer was tasked to expose a GET method on a new `/getcourses` resource to invoke the Lambda function, which will allow the consumers to fetch a list of online courses in JSON format. The consumers must include a query string parameter named `courseType` in their request to get the data.

What is the MOST efficient solution that the developer should do to accomplish this requirement?

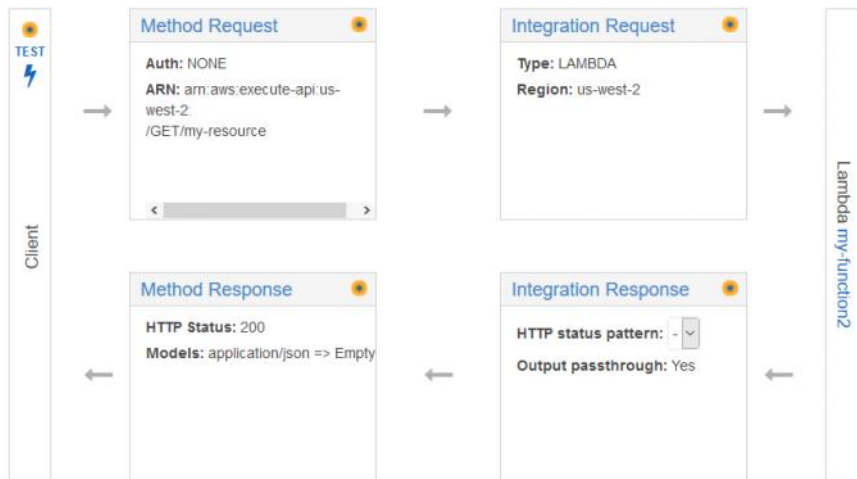
- Configure the integration request of the resource.
- Configure the method response of the resource.
- Configure the integration response of the resource.
- Configure the method request of the resource.

Incorrect

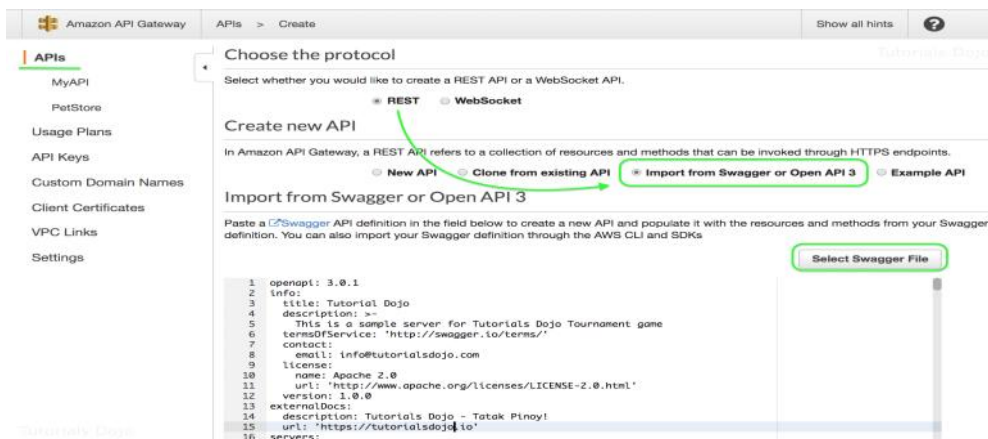
In Lambda non-proxy (or custom) integration, you can specify how the incoming request data is mapped to the integration request and how the resulting integration response data is mapped to the method response.

For an AWS service action, you have the AWS integration of the non-proxy type only. API Gateway also supports the mock integration, where API Gateway serves as an integration endpoint to respond to a method request. The Lambda custom integration is a special case of the AWS integration, where the integration endpoint corresponds to the [function-invoking action](#) of the Lambda service.

/my-resource - GET - Method Execution



The Lambda proxy integration type (`AWS_PROXY`) lets an API method be integrated with the Lambda function invocation action with a flexible, versatile, and streamlined integration setup. This integration relies on direct interactions between the client and the integrated Lambda function. With this type of integration, also known as the Lambda proxy integration, you do not set the integration request or the integration response. API Gateway passes the incoming request from the client as the input to the backend Lambda function.



In this scenario, you have to enforce the use of a required `courseType` query string parameter in the `/getcourses` resource in API Gateway. In order to do this, you can configure the method request of your resource just as shown in the diagram above.

Hence, the correct answer is to **configure the method request of the resource**.

Configuring the integration request of the resource is incorrect. Although configuring the *integration request* may also be valid, the client traffic will hit the *method request* first before it goes to the *integration request* down to the underlying Lambda function. This is why you should configure the *method request* first so it won't be necessary to check the required parameters in the Lambda integration. In addition, the *integration request* does not have the capability to enforce a request to include certain query string parameter nor enable API caching, unlike the *method request*.

Configuring the integration response of the resource is incorrect because you have to use the method request. Configuring the response, either the method-type or the integration-type, is irrelevant in this scenario.

Configuring the method response of the resource is incorrect because you should configure the method request instead. Take note that the scenario explicitly mentioned about the required query parameter which needs to be present before the processing can proceed.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-method-settings-method-request.html#setup-method-request-model>

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-integration-types.html>

<https://docs.aws.amazon.com/apigateway/latest/developerguide/set-up-lambda-proxy-integrations.html#api-gateway-simple-proxy-for-lambda-input-format>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

From <https://portal.tutorialsdojo.com/courses/aws-certified-developer-associate-practice-exams/lessons/practice-exams-section-based-3/quizzes/aws-certified-developer-associate-practice-exam-section-based-development-with-aws-services/>